



OneATS

User Manual

Confidential

A Strategic Partner Of



株式会社 東光高岳
TAKAOKA TOKO CO., LTD.



OneATS user manual

Copyright © 2021 Applied Technical Systems Joint Stock Company

Documents published and copyrighted by Applied Technical Systems Joint Stock Company (ATS JSC).

No part of this publication is permitted to be copied or published in any form or medium without prior written permission of Applied Technical Systems Joint Stock (ATS JSC).

We always look forward to receiving the comments of customers and readers for the purpose of perfecting the software and documents.

Any contributions, contact about software and documents please send to
ATS JSC: email: contact@ats.com.vn, website: <https://ats.com.vn/contact>

Definitions, symbols, abbreviations

Symbol	Description
CIM	Common Information Model
DMS	Distributed Management System
DCOM	Distributed Component Object Model
FEP	Front End Processing
HMI	Human Machine Interface
I/O	Input/Output
IED	Intelligent Electronic Device
LAN	Local Area Network
OPC UA	OPC Unified Architecture
RTU	Remote Terminal Unit
SCADA	Supervisory Control and Data Acquisition
SSL	Secure Socker Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
XML	Extensible Markup Language

Text color:

Comment

Notes

Warnings

Content

Chapter	Content
A.	System overview
B.	Database
C.	OneATS DataEditor – Project Management
D.	OneATS FEP
E.	OneATS DataServer
F.	OneATS Grid Studio
G.	OneATS SmartHIS
H.	OneMasterTool



OneATS User Manual

- OneATS DataServer

26/Jun/2024

.....

.....

.....

.....

.....

.....



Software Version	Date of Issue	Change Description
V4.2.3.6	May 15, 2024	

Table of contents

E. OneATS DataServer	E-1
<i>E.1. Overview OneATS DataServer.....</i>	<i>E-2</i>
E.1.1. OneATS DataServer Architecture	E-2
<i>E.2. Install and configure the application.....</i>	<i>E-3</i>
E.2.1. Install the app	E-3
E.2.2. Install and check your license	E-4
E.2.3. App Configuration	E-4
<i>E.3. Data Configuration.....</i>	<i>E-5</i>
E.3.1. Facility Declaration	E-5
E.3.2. DataModel Data Configuration.....	E-16
E.3.3. Data Mapping	E-34
E.3.4. Template	E-39
E.3.5. Variables Node	E-48
E.3.6. Alarm Configuration	E-50
E.3.7. Import DataModel.....	E-52
E.3.8. Export DataModel	E-52
E.3.9. Some things to do when working with a DataModel	E-53
<i>E.4. Configure runtime deployments</i>	<i>E-56</i>
E.4.1. Configure DataServer services.....	E-56
E.4.2. Boot OneATS DataServer service with Windows.....	E-56
<i>E.5. Monitoring Log - runtime</i>	<i>E-57</i>
<i>E.6. Monitor runtime data</i>	<i>E-58</i>
<i>E.7. Switching Order Class.....</i>	<i>E-59</i>
<i>E.8. Data Retention Management.....</i>	<i>E-60</i>
<i>E.9. Programming in OneATS DataServer</i>	<i>E-61</i>
E.9.1. Types of Scripts.....	E-61
E.9.2. Scripting Languages in OneATS DataServer	E-61
E.9.3. Calling Method	E-61
E.9.4. Variables in the script.....	E-62
E.9.5. Data Model/Variable dotfield	E-62
E.9.6. Built-in function	E-63
E.9.7. Using arrays in programming.....	E-69

List of figures

Figure E-1. Declare Alarm Categories	E-5
Figure E-2. Declare Alarm Sounds.....	E-5
Figure E-3. Declare Alarm Groups.....	E-6
Figure E-4. Declaring the Binary Alarm Group	E-6
Figure E-5. Declare Discrete Alarm Groups.....	E-7
Figure E-6. Declare Limit Alarm Groups	E-7
Figure E-7. Declaration of Rate of Change Alarm Group	E-8
Figure E-8. Declare local variables	E-9
Figure E-9. User defined function declaration	E-11
Figure E-10. Digital Set Declaration	E-12
Figure E-11. Declared Voltage Level	E-13
Figure E-12. Switching Map Declaration	E-14
Figure E-13. Load Distribution Configuration.....	E-15
Figure E-14. Configuring a region node.....	E-16
Figure E-15. Region deletion menu	E-17
Figure E-16. Add new sub region menu	E-17
Figure E-17. Sub region deletion menu	E-17
Figure E-18. New Add Menu Equipment Container	E-18
Figure E-19. Equipment Container Removal Menu	E-18
Figure E-20. New Add Menu Voltage level.....	E-19
Figure E-21. Voltage level removal menu	E-19
Figure E-22. New Add Menu Bay	E-20
Figure E-23. Bay Delete Menu	E-20
Figure E-24. Add new object menu	E-21
Figure E-25. Delete node object	E-21
Figure E-26. Program content editor window.....	E-23
Figure E-27. Method content editor window	E-25
Figure E-28. Delete method	E-25
Figure E-29. Add a new variable menu.....	E-26
Figure E-30. Variable node configuration.....	E-26
Figure E-31. Variable removal menu	E-29
Figure E-32. Add a new node from the template menu.....	E-30
Figure E-33. List of templates	E-31
Figure E-34. Configuring a node from a template	E-31
Figure E-35. Delete a build node from a template	E-31
Figure E-36. Exclude nodes in the datamodel.....	E-32
Figure E-37. Tagging Configuration	E-32
Figure E-38. Alarm category general configuration.....	E-33
Figure E-39. Data mapping.....	E-34
Figure E-40. Switching map.....	E-36

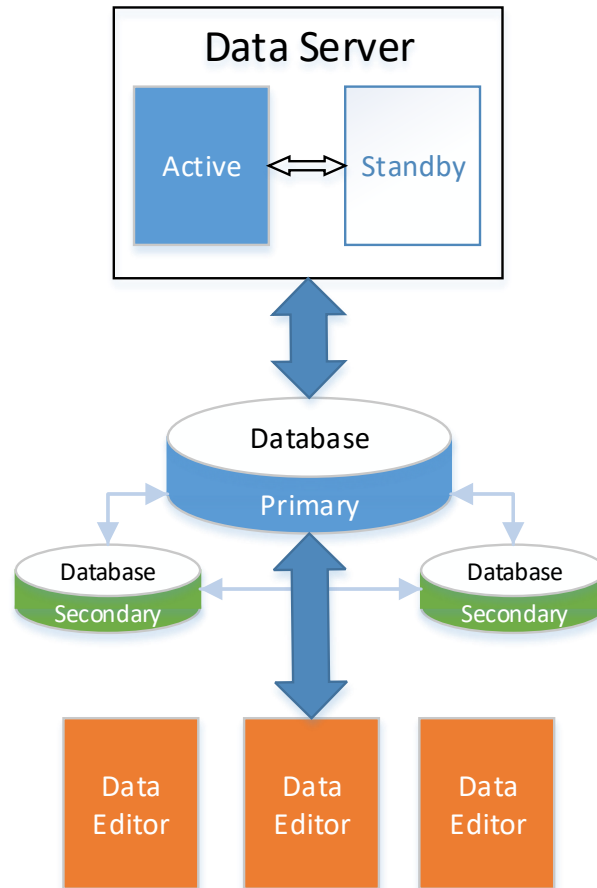
Figure E-41. Using Compute Blocks	E-37
Figure E-42. Script-style data mapping.....	E-38
Figure E-43. Method mapping for control points	E-38
Figure E-44. Build different objects from the template	E-39
Figure E-45. Manual template builder menu	E-40
Figure E-46. Interface Object Template builder.....	E-41
Figure E-47. Variable declaration menu	E-44
Figure E-48. Variable Configuration.....	E-44
Figure E-49. Menu to open template	E-45
Figure E-50. Template import menu	E-47
Figure E-51. Select a template file to import	E-47
Figure E-52. Menu export template	E-47
Figure E-53. Choose where to save your template.....	E-48
Figure E-54 Variable Data type	E-49
Figure E-55. Principle of alarm generation.....	E-50
Figure E-56. Make a Copy Interface on DataModel.....	E-54
Figure E-57. OneATS DataServer application log file	E-57
Figure E-58. DataServer runtime data monitoring	E-58
Figure E-59. Data Monitoring on HIServer	E-58
Figure E-60. Data Retention Management	E-60

E. OneATS DataServer

OneATS DataServer is a computer program that operates in the background (service) mode, acting as a central component of the system. Responsible for managing and processing all real-time data of the system, with input data from the OPC UA Server, including **OneATS FEP**.

E.1. Overview OneATS DataServer

E.1.1. OneATS DataServer Architecture



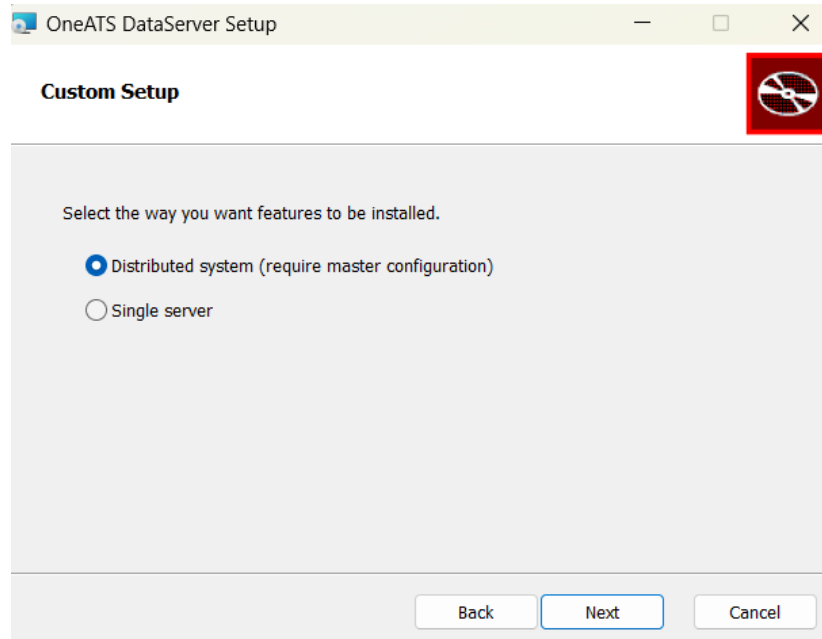
The figure above shows the relationship between the DataServer and the database editor. In which, the database system plays the role of a place to store the entire database of the system (including: data collected from IEDs, or stations, data exchanged with the SCADA system of the registration center, connected communication channels, etc.). All changes and updates to the database on the SCADA system will be made on this single database and the changed content will be automatically updated synchronously to the application function modules of the SCADA software including: HMI, HIS, FEP.

DataServer is a computer program that operates in the background (Service), plays a role as a central component in the SCADA system, is a bridge between FEP modules and other applications (HMI, HIS, etc.), which is responsible for managing and processing all real-time data of the system. Data mapping, soft interconnection, control processes, and warning thresholds after being configured with the DataEditor will be executed on the DataServer.

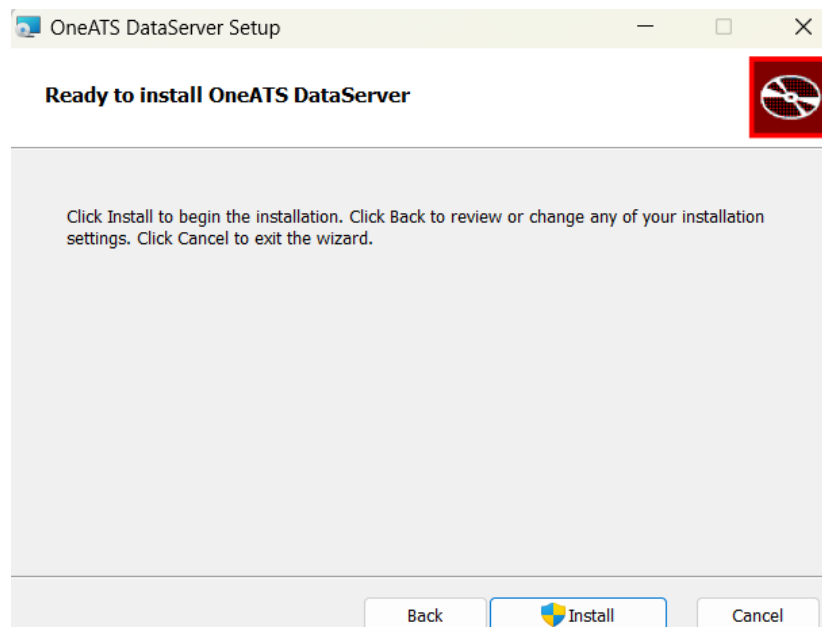
E.2. Install and configure the application

E.2.1. Install the app

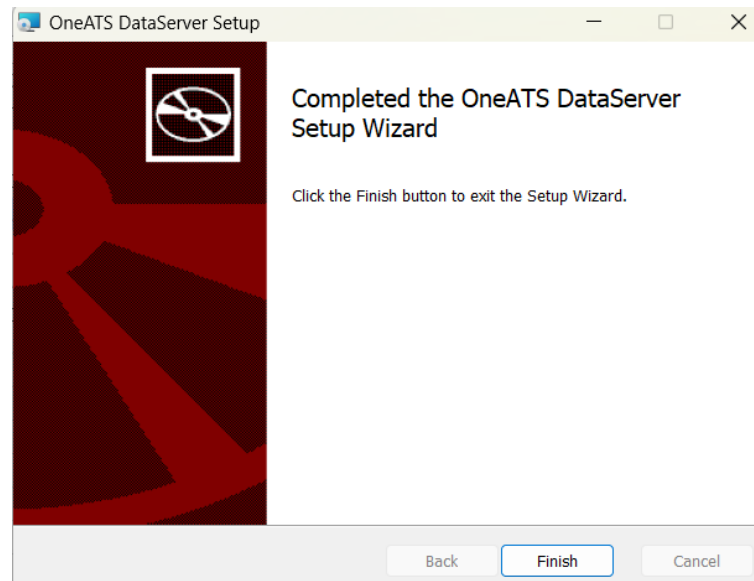
Run the ATS OneATS DataServer.msi **installation file**. The settings interface appears, select **Next** to continue:



Select **Install** and wait for the installation to take place:



Select **Finish** to finish the installation process:



E.2.2. Install and check your license

Refer to the [ChapterH.OneMasterTool](#) documentation

E.2.3. App Configuration

Refer to the [ChapterH.OneMasterTool](#) documentation

E.3. Data Configuration

E.3.1. Facility Declaration

E.3.1.1 Declare Alarm Category

In the OneATS DataEditor, select **Model/Configuration/Alarm Category** to open the **Alarm Category Configuration** window

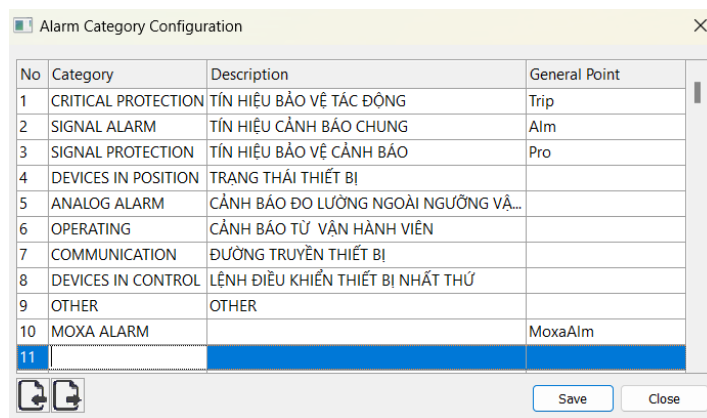


Figure E-1. Declare Alarm Categories

Double-click on the blank line next to the **Category** column, fill in the name for the **Category**, description in the **Description** and **General Point** columns (**General Point** Configuration refer to E.3.2.14) and then click **Save**.

Alarm Categories can be imported, exported from/out of excel files in .xlsx format.

E.3.1.2 Alarm Sound Declaration

From the menu, select **MODEL/Configuration/Alarm Sound** to open the **Alarm Sound Configuration** window.

Here, fill in the names for the audio files in the **Sound** column and click **Save** to finish.

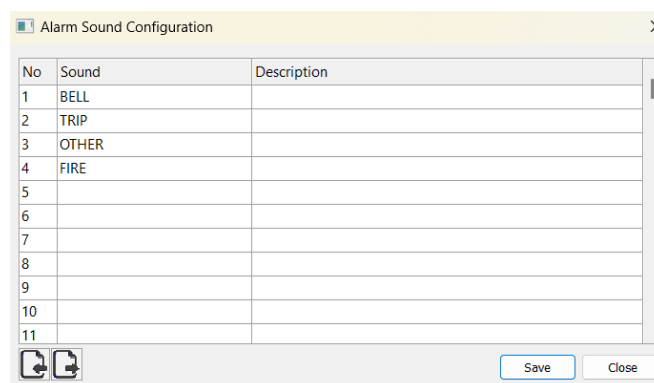


Figure E-2. Declare Alarm Sounds

Alarm Sounds can be imported, exported from/out of excel files in .xlsx format.

E.3.1.3 Alarm Group Declaration

In the **OneATS DataEditor**, select **Model/Configuration/Alarm Group** to open the **Alarm Group Configuration** window.

Declare the alert group by **Right Click** select **New**, enter the Group name, select the **Alarm** type, and then press **OK**.

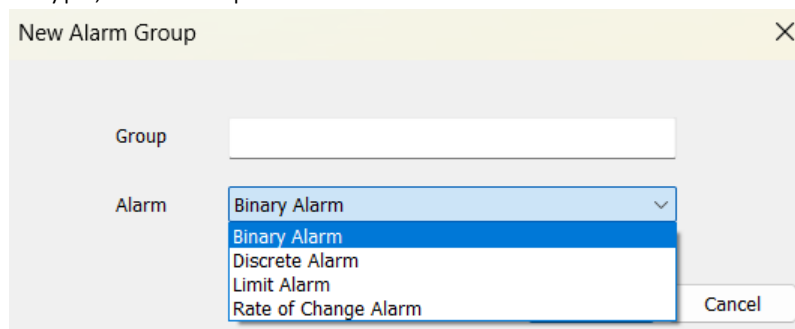


Figure E-3. Declare Alarm Groups

E.3.1.3.1 Configuring Binary Alarm Groups

Click select Group, enter **a comment**, select the **Category** style, enter the **Severity** value, select **the Sound** style, and then click **Save** to save.

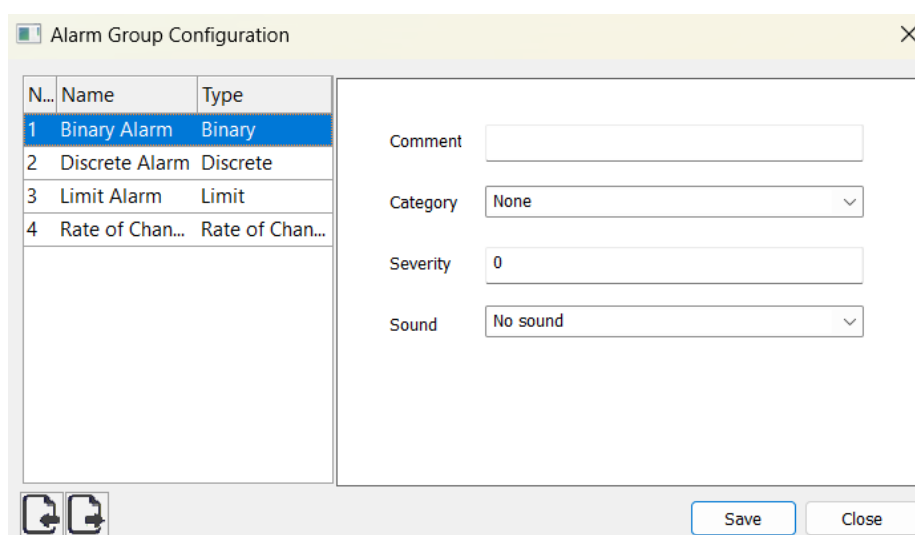
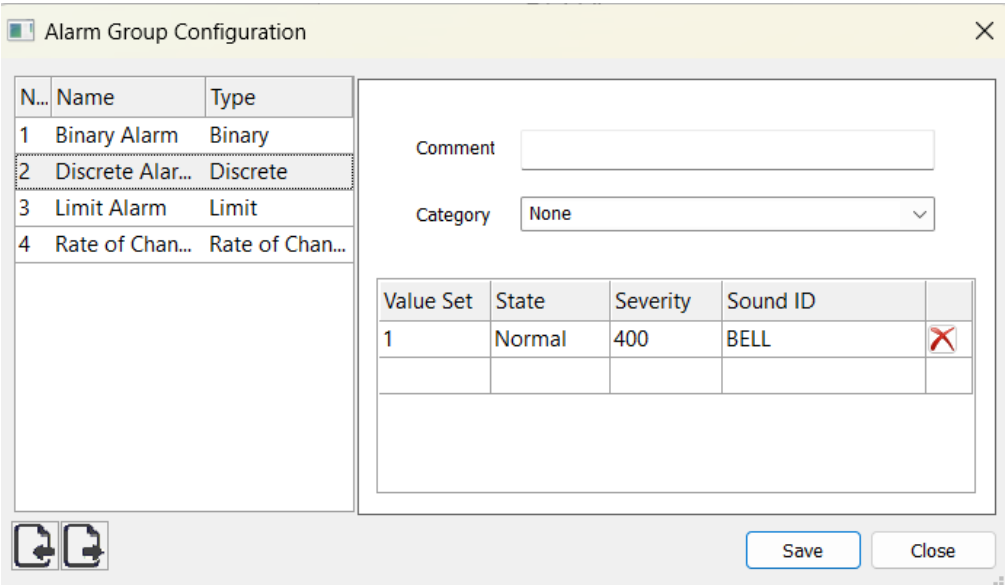


Figure E-4. Declaring the Binary Alarm Group

E.3.1.3.2 Configure discrete alarm group

Click select Group, enter **a comment**, select **a Category** style, enter **a Value Set** value, select **a State** style, enter **a Severity** value, select **a Sound ID** style, and then click **Save** to save.



The 'Alarm Group Configuration' window shows a list of alarm types on the left. The 'Discrete Alarm' group is selected. On the right, there is a 'Comment' text field, a 'Category' dropdown menu set to 'None', and a table for defining alarm values.

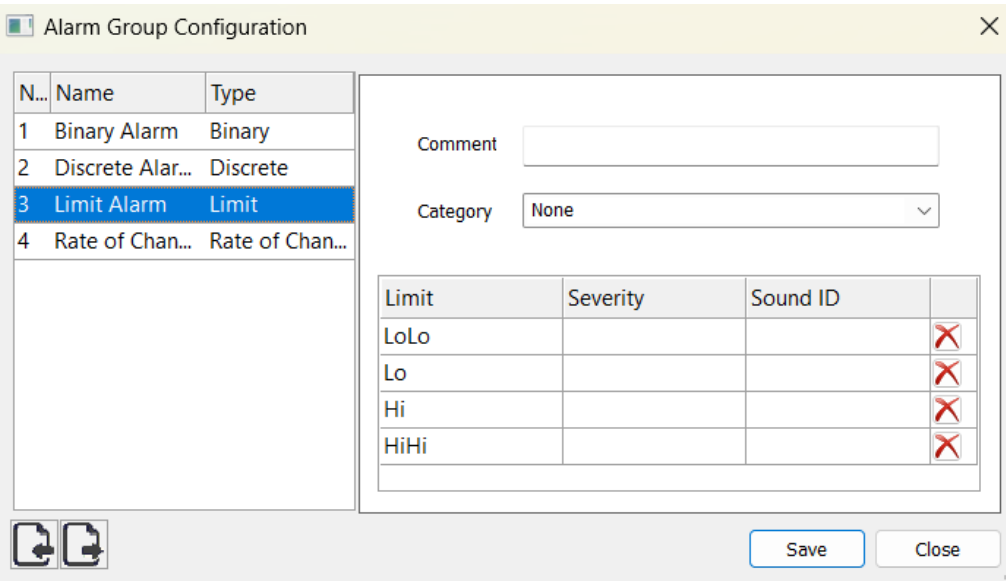
Value Set	State	Severity	Sound ID	
1	Normal	400	BELL	X

At the bottom are 'Save' and 'Close' buttons.

Figure E-5. Declare Discrete Alarm Groups

E.3.1.3.3 Limit alarm group configuration

Click select Group, enter a comment, select the Category type, enter the Severity value, select the Sound ID type , and then click Save to save.



The 'Alarm Group Configuration' window shows the 'Limit Alarm' group selected in the list. The right panel has the same 'Comment' and 'Category' fields. The table below is configured for limit alarms.

Limit	Severity	Sound ID	
LoLo			X
Lo			X
Hi			X
HiHi			X

At the bottom are 'Save' and 'Close' buttons.

Figure E-6. Declare Limit Alarm Groups

E.3.1.3.4 Configuring the rate of change alarm group

Click select Group, enter a comment, select the Category style, enter the Severity value, select the Sound style, and then click Save to save.

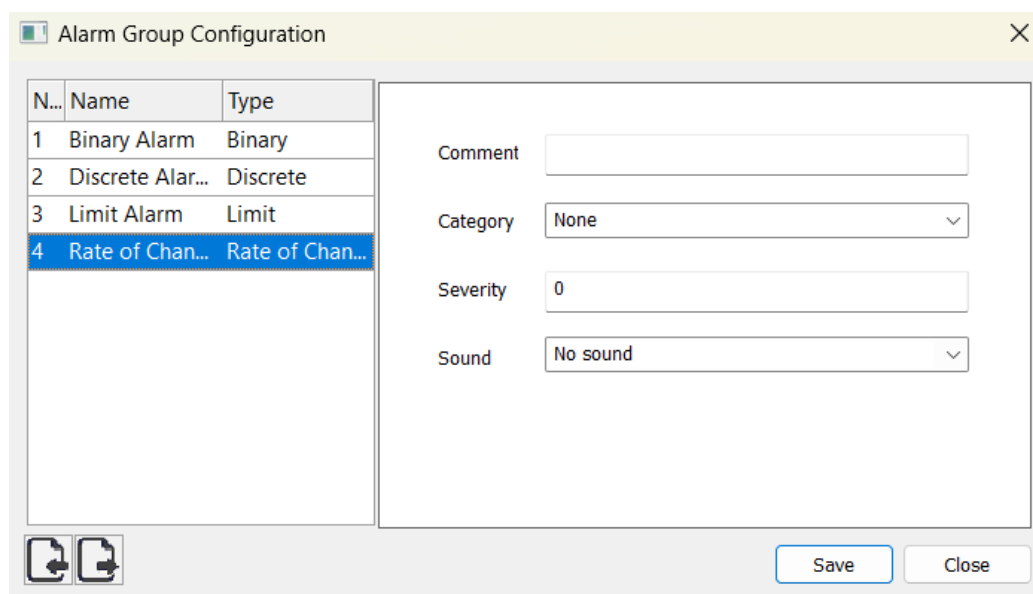


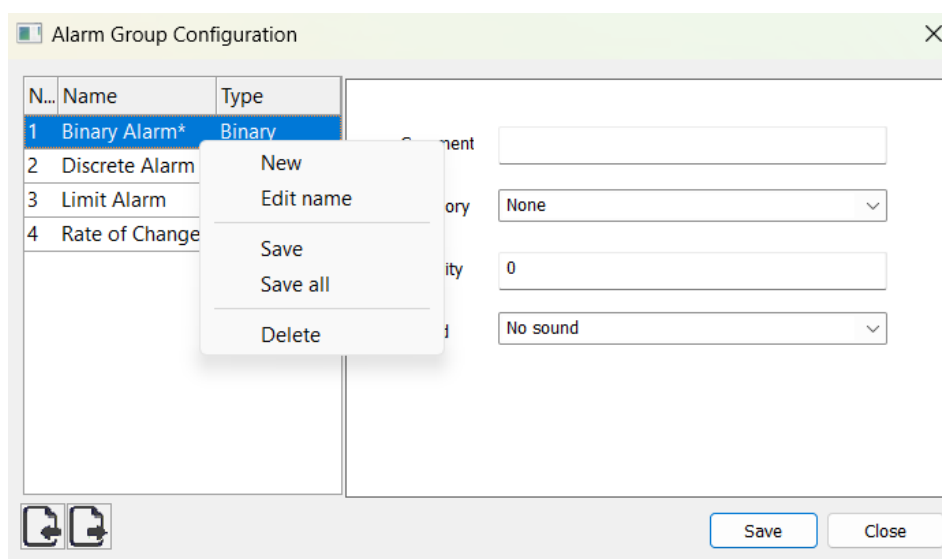
Figure E-7. Declaration of Rate of Change Alarm Group

Alarm Groups can import, export from/out excel files in .xlsx format.

Declare a warning group by **Right Click** select **New**

Some other operations with Group: Click to select Group, **Right Click** to select:

- **Edit Name:** Correct the name of the Alarm Group
- **Save:** Save the Alarm Group configuration
- **Save all:** Save the configuration of all Alarm Groups
- **Delete:** Delete Alarm Group



E.3.1.4 List of Local variables

Local variables include system variables and local variables.

From the menu, select **Model/Configuration/Local Variable**

Local Variable Configuration

Name	Data Type	Initial Value	TON Interval	Comment
_system_Database_address	String		0	
_system_Database_isPrimary	Boolean	False	0	
CBTm	Timer		10000	TIMER FOR CB CONTROLLING
ControlFailure	Boolean	False	0	CHECK TO SHOW CONTROL_FAIL...
CtlTm	Timer		2000	TIMER FOR FUNCTION ON/OFF
EmerTm	Timer		1000	
FanTm	Timer		5000	TIMER FOR FAN CONTROLLING
PanelMsg	String	PERMISSIO...	0	Message on Control Panel
PumpTm	Timer		5000	TIMER FOR PUMP CONTROLLING
SWTm	Timer		20000	TIMER FOR DS CONTROLLING
Subs_BeingCtl	Boolean	False	0	
Subs_CmdSuccess	Boolean	False	0	
Subs_ForceEnaSt	Boolean	False	0	
TapTm	Timer		9000	TIMER FOR TAP CONTROLLING
_system_null	Float		0	

Close

System variables are grayed out and cannot be changed.

Local variables are black and subject to change.

Right click on the local variable configuration area and select **New** to open the local variable configuration window.

Fill in the configuration parameters for the variable and press **Save** to finish.

New Local Variable

Name

Data Type

Initial Value

TON Interval ms

Comment

Save Cancel

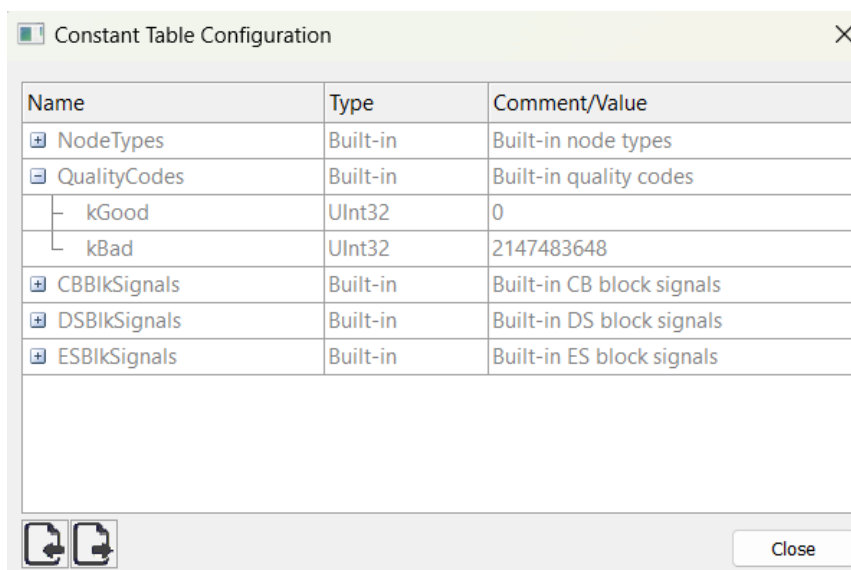
Figure E-8. Declare local variables

Right-click on the local variable, select **Edit** to open the editing window, or select **Delete** to delete the local variable, or Double-click on the local variable to **edit** the local variable.

Local variables can be imported, exported from/out of excel files in .xlsx format.

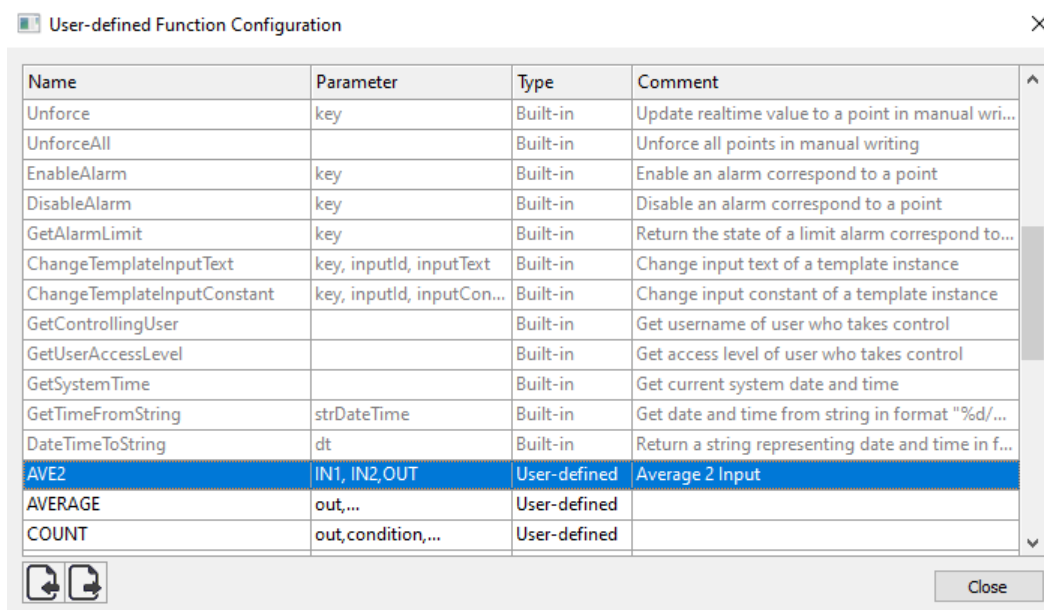
E.3.1.5 Constant Board List

From the menu, select **Model/Configuration/Constant Table**



E.3.1.6 List of User defined functions

From the menu, select **Model/Configuration/User-defined Function**



The System Function is grayed out and cannot be changed.

User-defined functions are black and subject to change.

Right-click on the User-defined Function configuration area and select **New** to open the Create New Function window.

Fill in the name, parameter list, description for the function.

Write the function content, click **Compile** to compile the function.

Click **Save** to save.

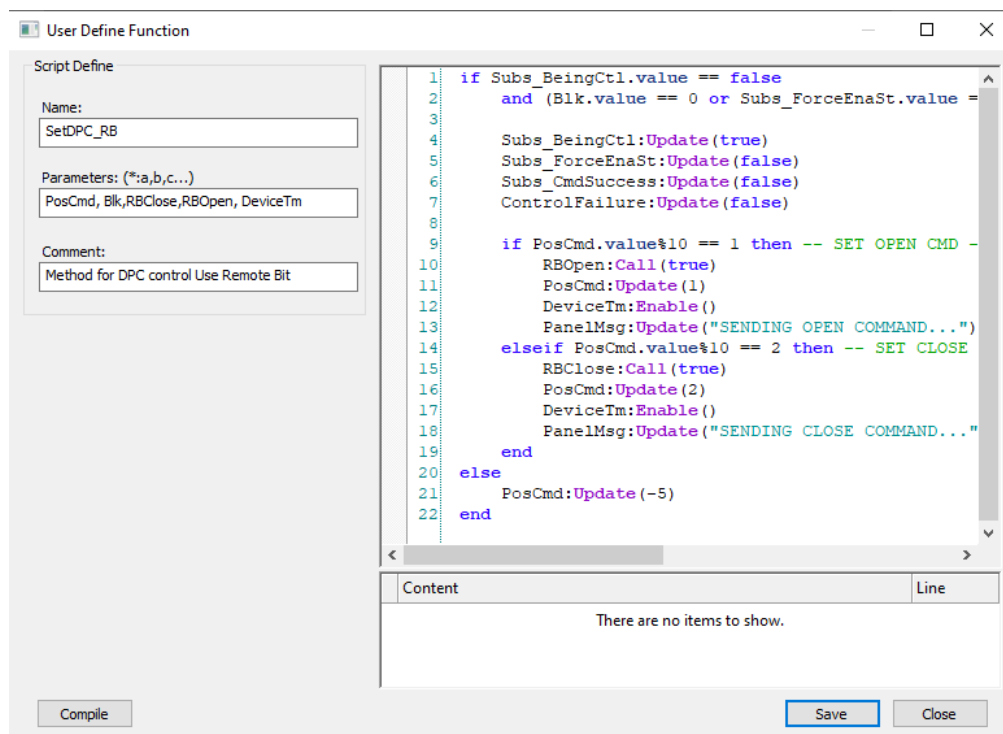


Figure E-9. User defined function declaration

Right-click on the User-defined Function, select **Edit** to open the editing window, or select **Delete** to delete the function, or Double-click on the User-defined Function to **Edit** the function.

The User defined Function can import, export from/out excel files in .xlsx format.

E.3.1.7 Digital Set Declaration

A **Digital Set** is a user-defined data type.

From the menu, select **Model/Configuration/Digital Set** to open the digital set configuration window.

Double-click on the blank line next to the **Digital Set** column, enter a name for the **Digital Set**, and select **Data Type**.

Double click on the line next to the **Enum String**, **Value** columns to add new or correct values.

Click **Save** to save.

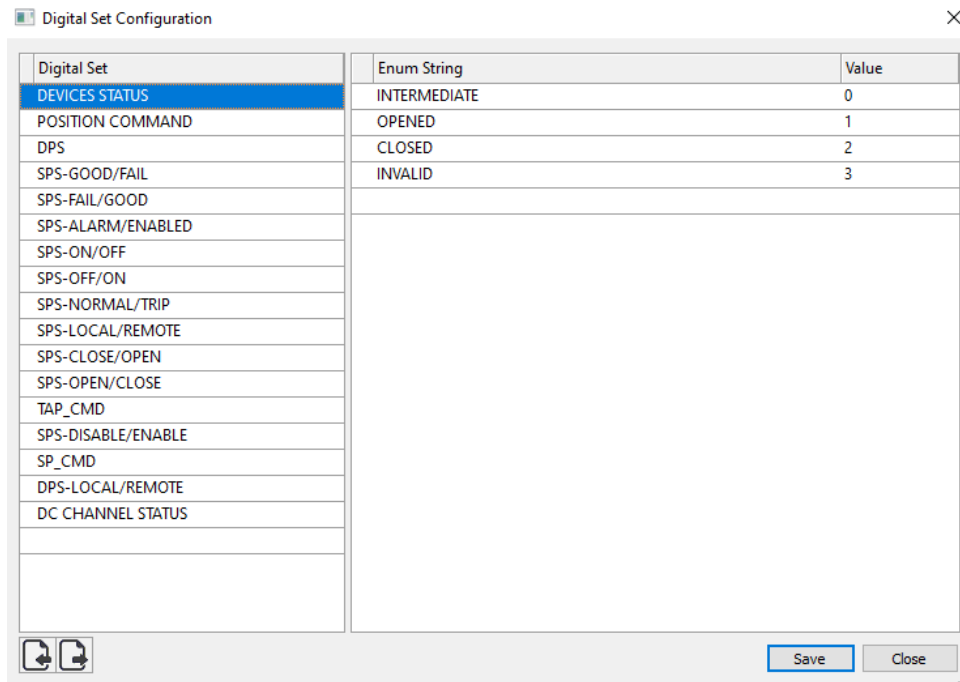


Figure E-10. Digital Set Declaration

Right Click on the Digital Set, select **Delete** to delete the Digital Set, or Double Click on the Digital Set to **Edit** the Digital Set.

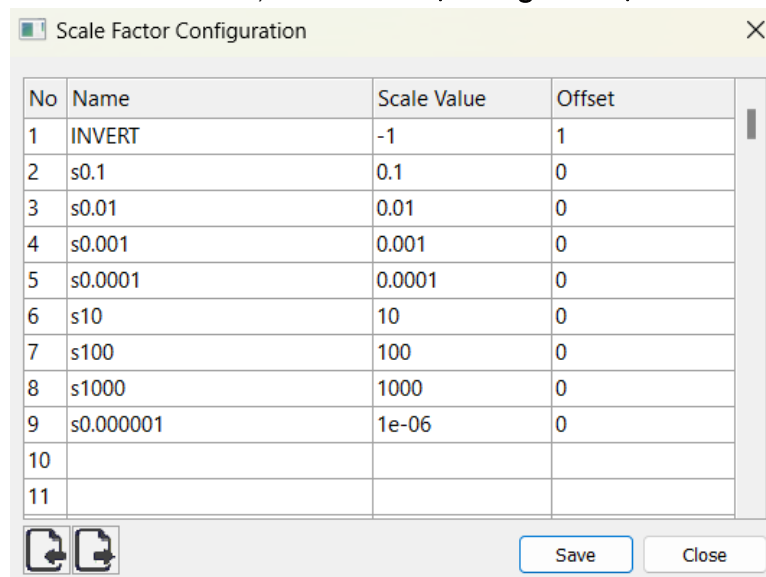
Digital Sets can be imported, exported from/out excel files in .xlsx format.

E.3.1.8 Scale Factor Declaration

Used in **Point-to-Point** mapping, the formula:

$$\text{OutValue} = \text{InValue} * \text{ScaleValue} + \text{Offset}$$

To declare a **Scale Factor**, select **Model/Configuration/Scale Factor**



Double Click on a line to add a new or correct **Scale Factor**.
Fill in the Name column, fill in the **Scale value**, Offset values respectively.
Click **Save** to save.

Scale Factors can be imported, exported from/out of excel files in .xlsx format.

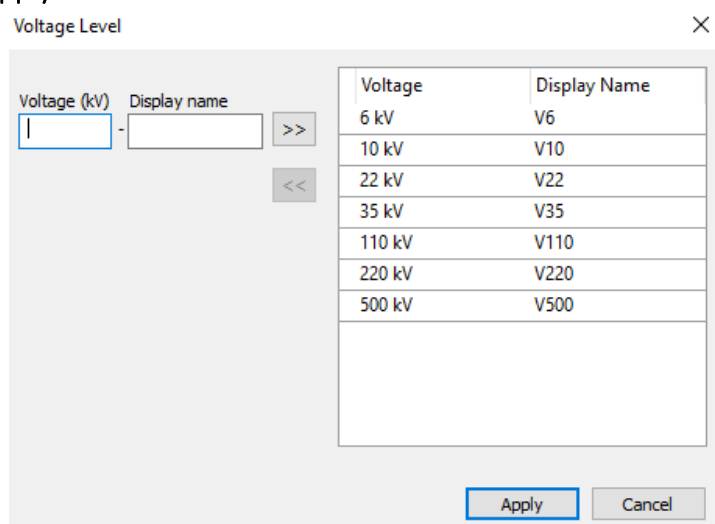
E.3.1.9 Declared Voltage Level

From the menu, select **Model/Configuration/Voltage Level**.

In the **Voltage Level window**, fill in the Voltage level in the **Voltage (kV) box**, enter the name displayed on the DataModel in the **Display name box**, and press **>>** to add a new voltage level.

To delete a Voltage Level, click select Voltage Level, press **<<**.

Click **Apply** to save.



Voltage	Display Name
6 kV	V6
10 kV	V10
22 kV	V22
35 kV	V35
110 kV	V110
220 kV	V220
500 kV	V500

Figure E-11. Declared Voltage Level

E.3.1.10 Declaring an Area of Responsibility (AOR)

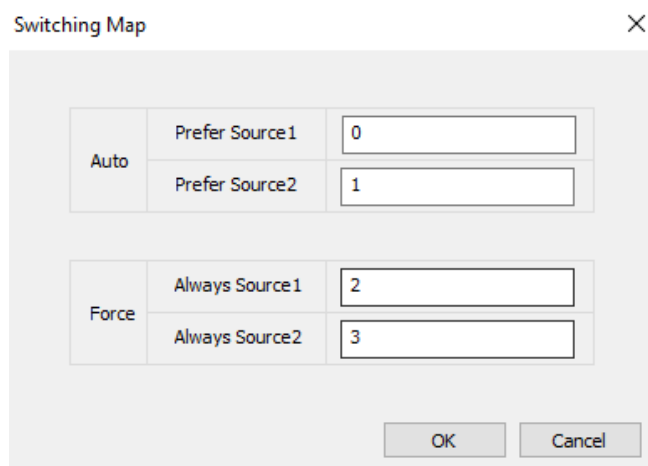
Refer to the **ChapterC.OneATS DE-Project Management** user manual.

E.3.1.11 AOR Group Declaration

Refer to the **ChapterC.OneATS DE-Project Management** user manual.

E.3.1.12 Switching Map Declaration

From the menu, select **Model/Configuration/Switching Map**, which serves data source transfer for datapoints with the Point to Point (2-Source) data map type.

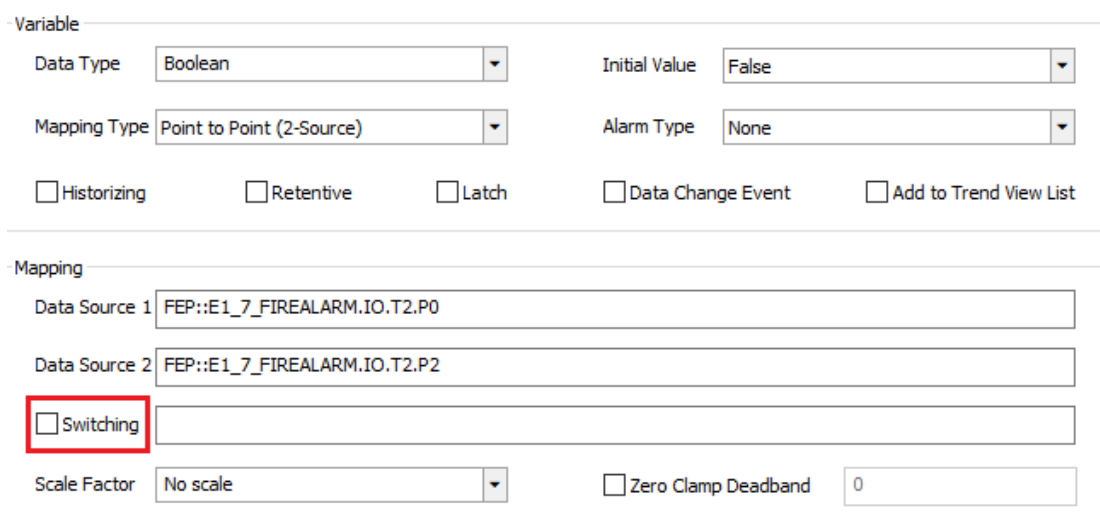


The dialog box titled "Switching Map" contains two main sections. The first section, labeled "Auto", has two rows: "Prefer Source1" with a text box containing "0", and "Prefer Source2" with a text box containing "1". The second section, labeled "Force", also has two rows: "Always Source1" with a text box containing "2", and "Always Source2" with a text box containing "3". At the bottom right are "OK" and "Cancel" buttons.

Figure E-12. Switching Map Declaration

The mechanism for transferring datasource sources is as follows:

- Mechanism to automatically switch datasource (Variable is not checked):
When starting the program, it will scan 2 sources in the order of Source 1, Source 2 and the datapoint will be mapped with a source with GOOD quality. During the runtime runtime, if the source being mapped with quality is changed to BAD, the program will consider whether the other source is having GOOD quality or BAD, if the other source has GOOD quality, it will map according to that source, and the other source has BAD quality, there will be no change. The program will scan 2 sources until it receives GOOD quality from either source.



The form is divided into two main sections: "Variable" and "Mapping".
 In the "Variable" section:
 - "Data Type" is set to "Boolean".
 - "Initial Value" is set to "False".
 - "Mapping Type" is set to "Point to Point (2-Source)".
 - "Alarm Type" is set to "None".
 - There are five checkboxes: "Historizing", "Retentive", "Latch", "Data Change Event", and "Add to Trend View List", all of which are currently unchecked.
 In the "Mapping" section:
 - "Data Source 1" is set to "FEP::E1_7_FIREALARM.IO.T2.P0".
 - "Data Source 2" is set to "FEP::E1_7_FIREALARM.IO.T2.P2".
 - The "Switching" checkbox is checked and highlighted with a red box.
 - "Scale Factor" is set to "No scale".
 - "Zero Clamp Deadband" is set to "0".

- Mixed mechanism (Variable is selected Switching): If the Switching variable has a value within the Auto range, the selection of the data source will comply with the automatic switching mechanism of the datasource, but the Switching value corresponds to which source, that source will be selected

first. If the Switching variable has a value in the Force range, then the data source selection is based on the Switch variable value and not on the quality of Source 1 or Source 2. For example, if the variable value Switching = 2, then Source 1 will be mapped with the datapoint, and if the variable value Switching = 3, then Source 2 will be mapped with the datapoint.

Variable

Data Type: Initial Value:

Mapping Type: Alarm Type:

☐ Historizing ☐ Retentive ☐ Latch ☐ Data Change Event ☐ Add to Trend View List

Mapping

Data Source 1:

Data Source 2:

☒ Switching:

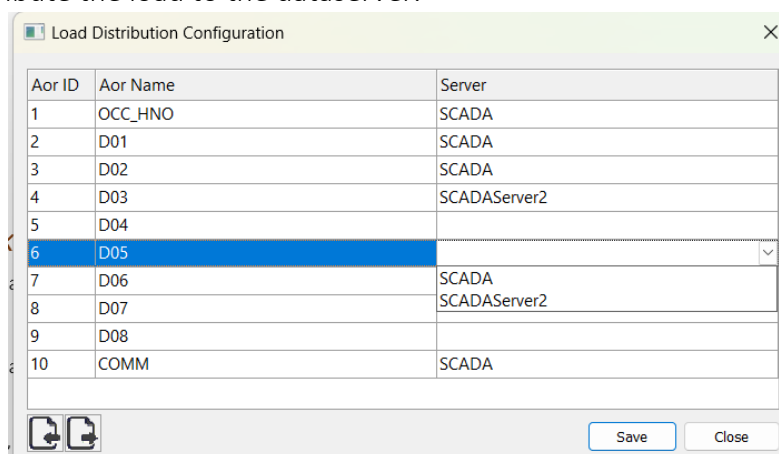
Scale Factor: ☐ Zero Clamp Deadband:

E.3.1.13 Topology

E.3.1.14 Choir Program

E.3.1.15 Load Distribution Configuration

From the menu, select **Model/Configuration/Load Distribution**, which serves to distribute the load to the dataserver.



Aor ID	Aor Name	Server
1	OCC_HNO	SCADA
2	D01	SCADA
3	D02	SCADA
4	D03	SCADAServer2
5	D04	
6	D05	
7	D06	SCADA
8	D07	SCADAServer2
9	D08	
10	COMM	SCADA

Save Close

Figure E-13. Load Distribution Configuration

Double-click on the **Server column** and select the server name in the list declared in the SCADA System for AORs.

Click **Save** to save.

Load Distribution configuration can import, export from/out excel files in .xlsx format.

E.3.2. DataModel Data Configuration

DataModel data is declared and managed in the **Model Explorer area** on the right side of the **OneATS DataEditor interface**.

The data group includes:

Region (Hostcontrol Area, Subcontrol Area, ...)

Equipment Container (Plant, Line, Substation, Voltage Level, Bay, ...)

Equipment (Circuit Breaker, Disconnecter, Busbar, Transformer, AC Line Segment, ...)

Object (Measurement, Protection, Device, ...)

Method (Control Command, ...)

Programs (Interlock, ResetLatch, CmdSPC, CmdDPC, ...)

Variable (Signals,)

E.3.2.1 Region node

E.3.2.1.1 Add a new Region

From the **Model Explorer window**, right-click on **Root**, select **Add Node** or select **Root**, then press **ctrl + N**, select **Region**

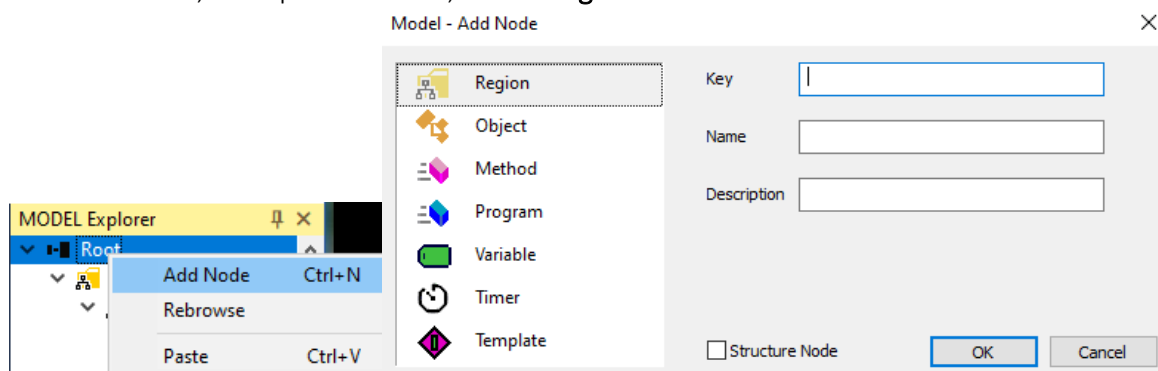


Figure E-14. Configuring a region node

Fill in the **Key**, **Name**, **Description** for the region you just created.
Press **Ok** to save.

E.3.2.1.2 Delete Region

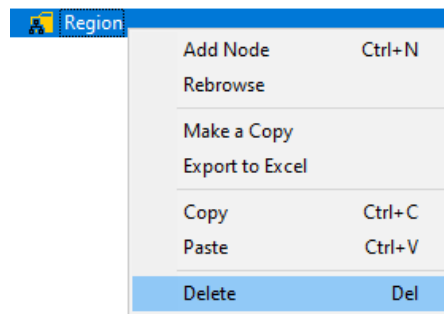


Figure E-15. Region deletion menu

Right-click on the **Region** you want to delete, select **Delete** or select **Region**, and then press the delete key.

E.3.2.2 Sub Region node

E.3.2.2.1 Add a new Sub Region

From the **Model Explorer** window, right-click on **Region** and select **Add Node** or select **Region** and then press **Ctrl + N**, select **Sub Region**

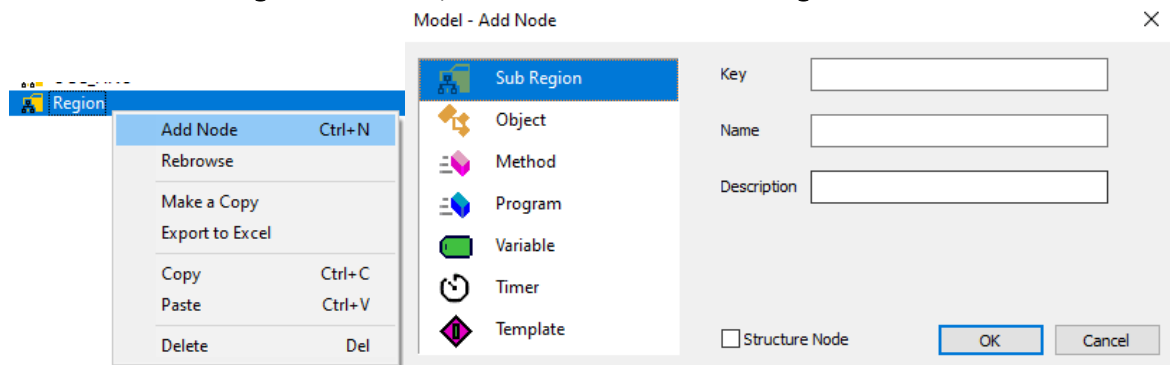


Figure E-16. Add new sub region menu

Fill in the **Key**, **Name**, **Description** for the sub region you just created. Press **Ok** to save.

E.3.2.2.2 Delete Sub Region

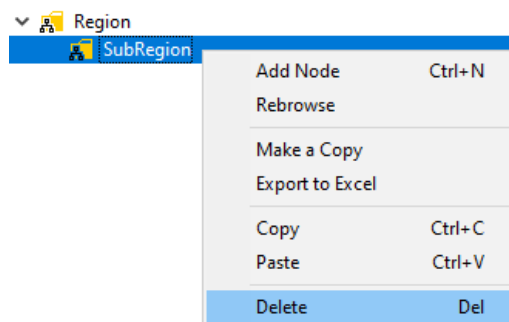


Figure E-17. Sub region deletion menu

Right click on the **Sub Region** you want to delete, select **Delete** or select **Sub Region**, then press the **delete key**.

E.3.2.3 Equipment container node

E.3.2.3.1 Add New Container Equipmen

From the **Model Explorer** window, right-click on **Sub Region** and select **Add Node** or select **Sub Region**, then press **ctrl + N**, select **Equipment Container**

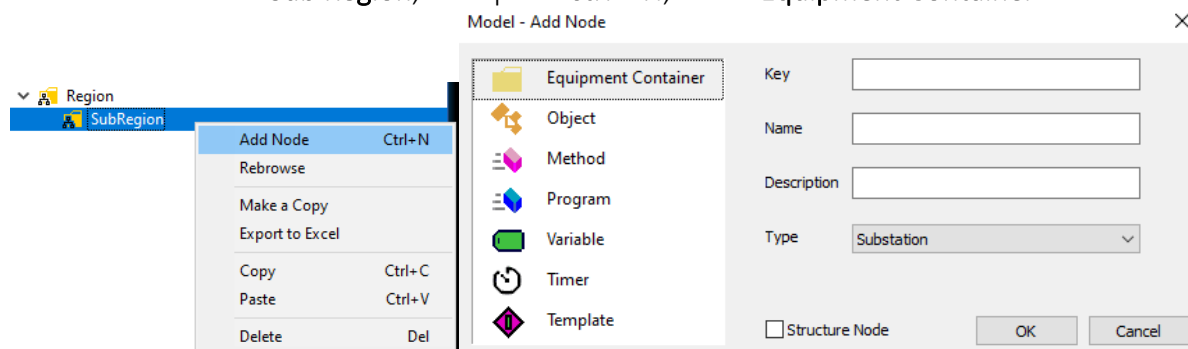


Figure E-18. New Add Menu Equipment Container

Fill in the **Key**, **Name**, **Description**, **Type** for the equipment container you just created.

Press **Ok** to save.

E.3.2.3.2 Delete equipment container

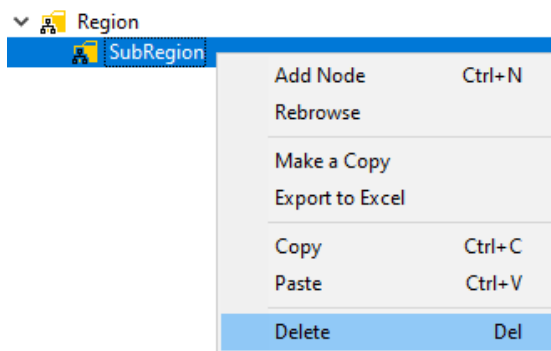


Figure E-19. Equipment Container Removal Menu

Right click on the **Equipment Container** you want to delete, select **Delete** or select **Equipment Container** and press the **delete key**.

Add and delete other nodes that do the same.

Structured Node: is structured data, when a node is declared as a Structured Note, the key of that node is automatically generated based on the declared name.

E.3.2.4 Voltage Level

E.3.2.4.1 Add a new Voltage level

From the **Model Explorer** window, right-click on **Equipment Container**, select **Add Node**, or select **Equipment Container**, and then click **ctrl + N**

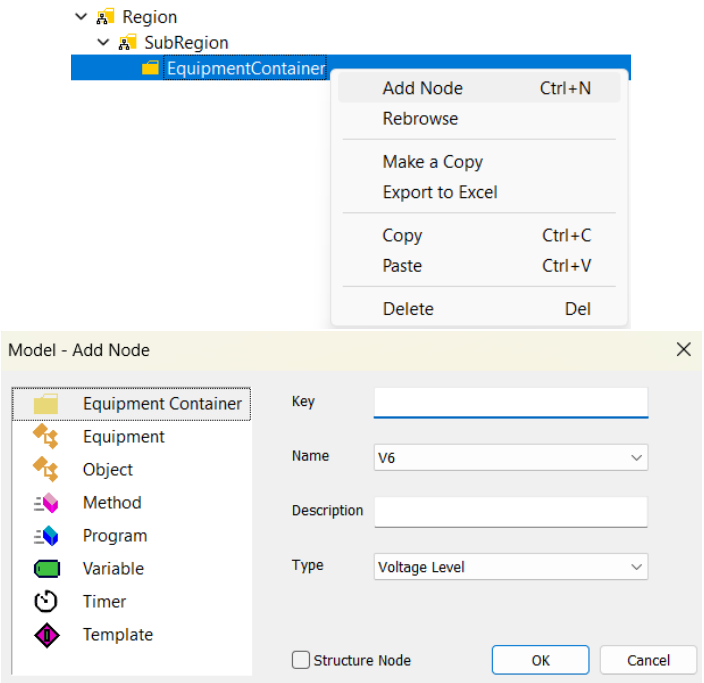


Figure E-20. New Add Menu Voltage level

Fill in **Key**, **Description**, select **Name** for the Voltage level you just created.
Press **Ok** to save.

E.3.2.4.2 Removing Voltage Level

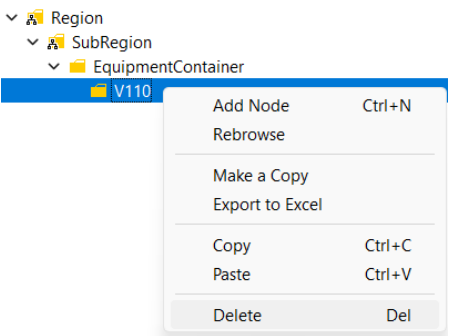


Figure E-21. Voltage level removal menu

Right click on the **Voltage level** you want to delete, select **Delete** or select **Voltage level** and press the **delete** key.

E.3.2.5 Bay nodes

E.3.2.5.1 Add New Bay

From the **Model Explorer** window, right-click on **Voltage Level** and select **Add Node** or Select **Voltage Level** and then press **ctrl + N**

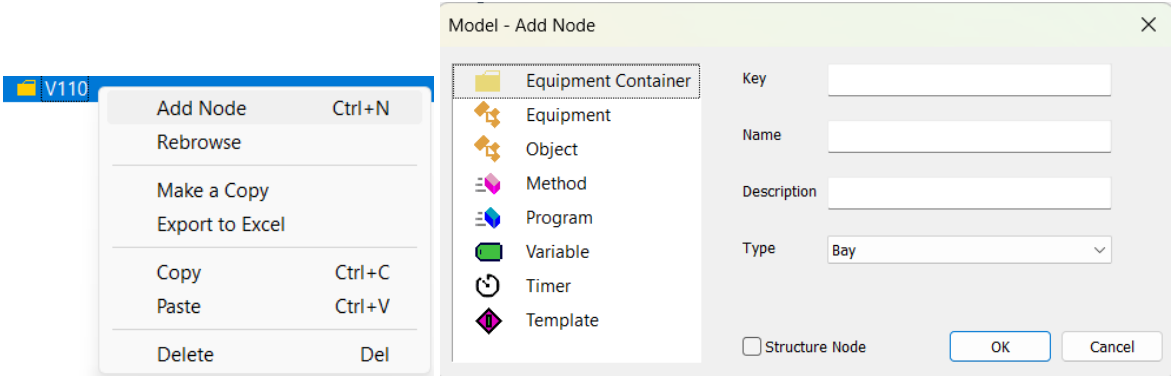


Figure E-22. New Add Menu Bay

Fill in the **Key**, **Name**, **Description** for the Bay you just created.
Press **Ok** to save.

E.3.2.5.2 Delete the Bay

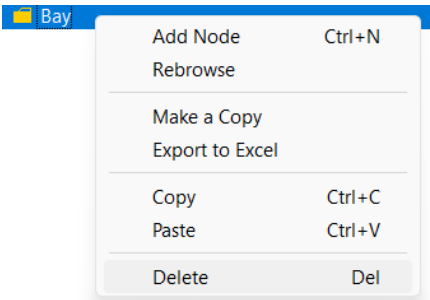


Figure E-23. Bay Delete Menu

Right-click on the **Bay** you want to delete, select **Delete** or select **Bay**, and then click the delete key.

E.3.2.6 Object node

E.3.2.6.1 Adding an Object

From the node you want to add an object to, right-click Add **Node** or press **ctrl + N**

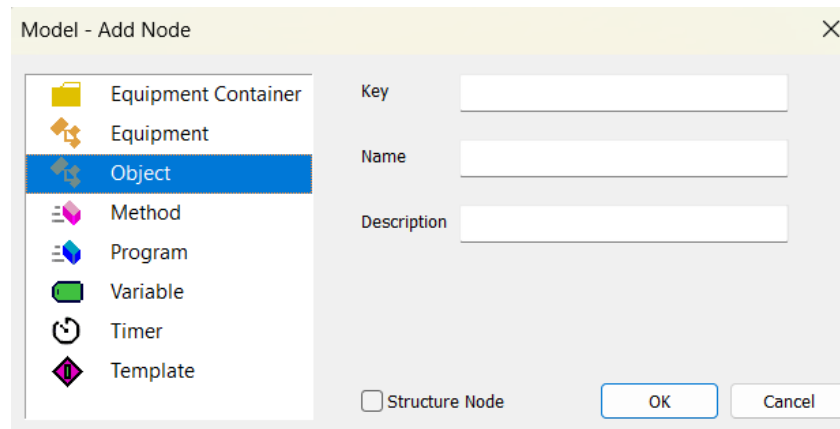


Figure E-24. Add new object menu

Fill in the **Key**, **Name**, **Description** for the object you just created.
Press **Ok** to save.

E.3.2.6.2 Object Deletion

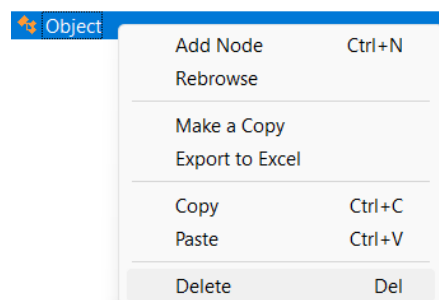


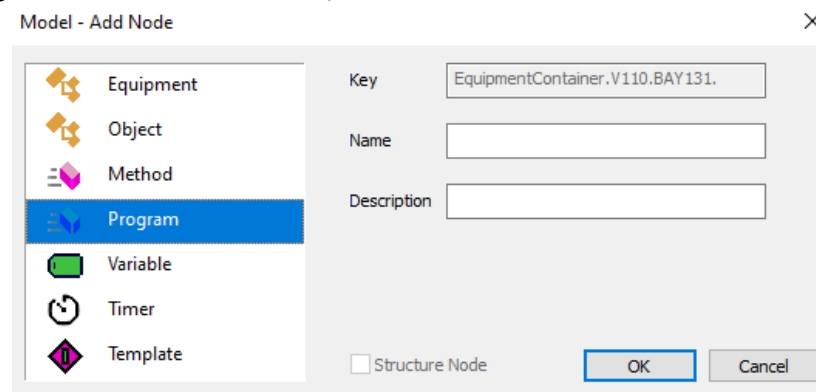
Figure E-25. Delete node object

Right-click on the **Object** you want to delete, select **Delete** or select the **Object** and then click the delete key

E.3.2.7 Node program

E.3.2.7.1 Add a Program

Right-click on the data node, select **Add Node** or click **Ctrl + N** to select **Program**.



Fill in the **Key**, **Name**, **Description** for the **Program**.

General

Key

E1_1.Program.CheckBusLineAlive

Type

Program

Name

CheckBusLineAlive

Area

D01

Description

Excluded

☐

Program

Enabled

☒

Mapping

Trigger by monitored items

Repeat with interval

1000 ms

Content

Edit

Script

Monitored Items

```

1  -- BAY100 --
2  if (MODEL::E1_1.V110.BAY100.MMXU1.Vlin.value > MODEL::COMM.
3      then MODEL::E1_1.V110.BAY100.BAY.IsLive:Update(true)
4  elseif (MODEL::E1_1.V110.BAY100.MMXU1.Vlin.value <= MODEL::
5      then MODEL::E1_1.V110.BAY100.BAY.IsLive:Update(false)
6  end
7  -- BAY112 --
8  if (MODEL::E1_1.V110.BAY112.MMXU1.Vlin.value > MODEL::COMM.
9      then MODEL::E1_1.V110.BAY112.BAY.IsLive:Update(true)
10 elseif (MODEL::E1_1.V110.BAY112.MMXU1.Vlin.value <= MODEL::
11     then MODEL::E1_1.V110.BAY112.BAY.IsLive:Update(false)

```

From the Mapping window, select the Program execution condition:

- **Trigger by monitored items:** Run the program when there is at least one item in the list of monitored items with data changes.
- **Repeat with interval:** Run the program periodically after the interval period.

Select **Edit** to open the Program content editing window.

Write the content for the program, press **Compile**, if the compilation process is error-free, a Succeeded message will appear in the Content section.

After **compilation**, the variables used in the **Program** content are listed in the **Monitored Items** window. For Programs of the **Trigger by monitored items** type, check the variables that are the conditions for program execution. Users can add an item to the **Monitored Items** list by Right-clicking on the list and selecting Add custom node, filling in the name of the item and pressing **OK** to add it.

Press the **ctrl + S** key combination to save the newly created program node.

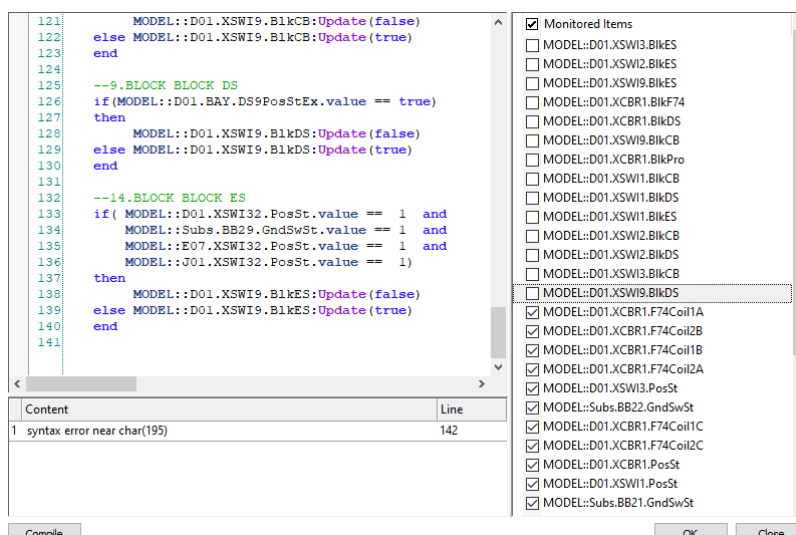
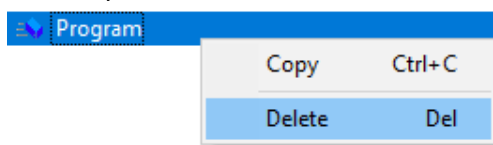


Figure E-26. Program content editor window

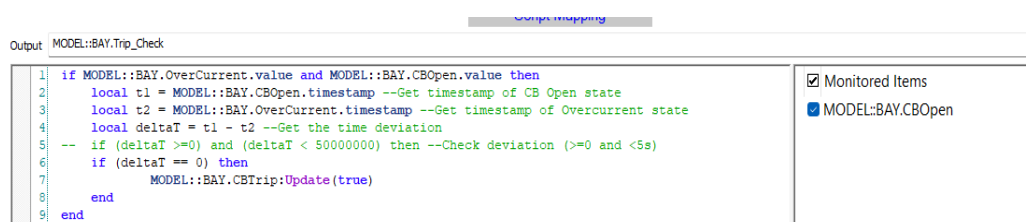
E.3.2.7.2 Delete Program

Right-click on the **Program** you want to delete, select **Delete** or select **Program** and then click the **delete** key.



E.3.2.7.3 Monitored Items

For programs that are set to Activated by Supervised Items mode, if an item in the list of supervised items is selected in the script editor window, the program will be executed when the item has a data change.

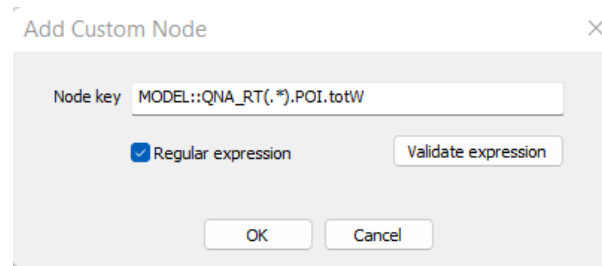


Users can add an item to **Monitored Items** by right-clicking on the list and selecting the **Add Custom** button, fill in the name of the item in the **OK** button to add it.

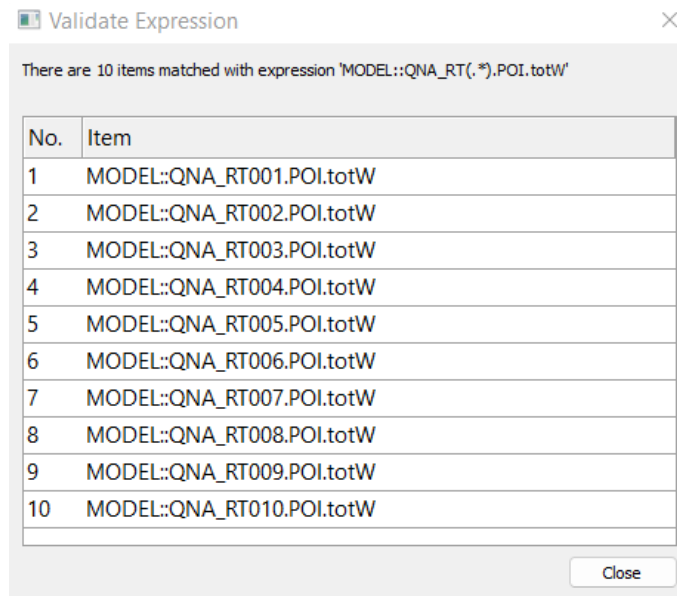
Custom buttons in Monitored Items can be items that are available in the model or can be Regular Expressions.

The Regular Expression type allows the use of alternate characters, which are used when it is necessary to keep track of all variables of the same type.

Example when filling in the Node key: MODEL::QNA_RT(*). POI.totWand selects Regular Expression, the program will keep track of all variables of the same format as the button key.



Users can click " **Validate expression** " to check items with the same format found in the database



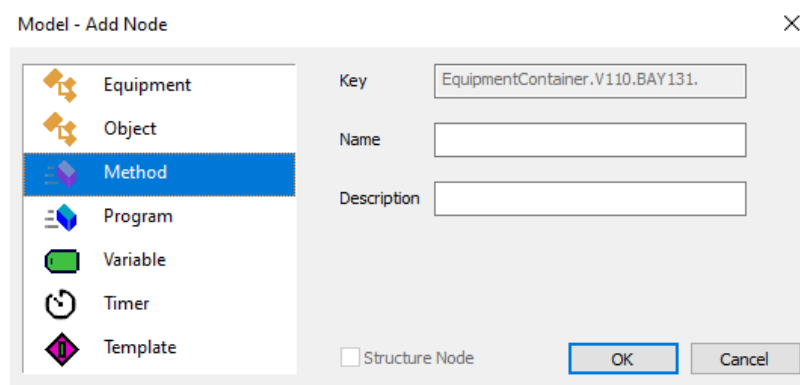
When a Node Key contains special characters, if you want to use a Regular Expression, you need to add a "\" before the Node key. For example, \ \$Key.INV(*).MMXU1.totW

For details, see: <https://m.cplusplus.com/reference/regex/ECMAScript/>

E.3.2.8 Method node

E.3.2.8.1 Add Method

Right-click on the data node, select **Add Node**, or click **ctrl + N** to select **Method**.



Fill in the **Name**, **Description** for the **Method**.

Declare the list of parameters in Parameters:

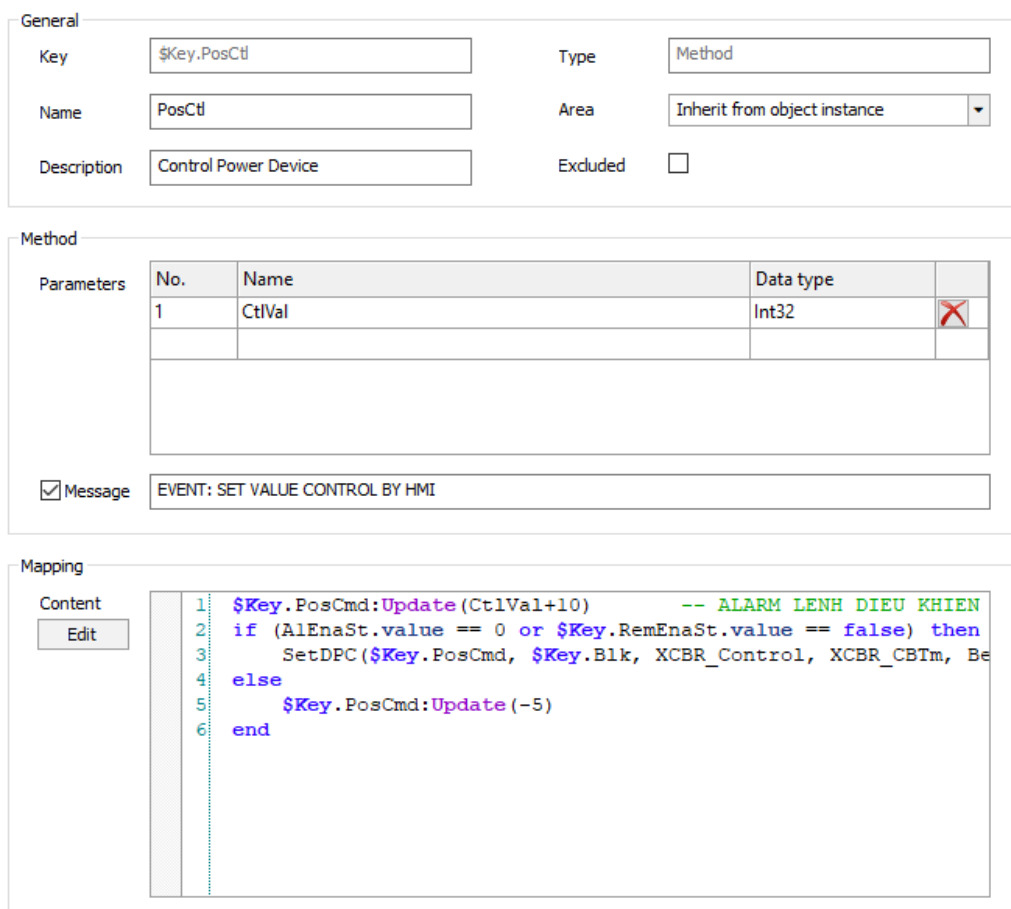
- Double-click on the blank line in the **Name** column to add a parameter name.
- Select the corresponding data type in the **Data type** column.

When a Message is selected, each time **the Method** is called, the system will generate an event with the content entered in the **Message** section.

Click **Edit** to open the Content Editor window for **the Method**.

To write the content for the method, click Compile. If the compilation process is error-free, a Succeeded message appears in the **Content** section.

Press **Ctrl + S** to save the newly created method.



The screenshot shows the 'Method content editor window' with three tabs: General, Method, and Mapping.

General Tab:

- Key: \$Key.PosCtl
- Type: Method
- Name: PosCtl
- Area: Inherit from object instance
- Description: Control Power Device
- Excluded: ☐

Method Tab:

No.	Name	Data type	
1	CtlVal	Int32	

☒ Message: EVENT: SET VALUE CONTROL BY HMI

Mapping Tab:

Content:

```

1  $Key.PosCmd:Update (CtlVal+10)           -- ALARM LENH DIEU KHIEN
2  if (AlEnaSt.value == 0 or $Key.RemEnaSt.value == false) then
3      SetDPC($Key.PosCmd, $Key.Blk, XCBR_Control, XCBR_CBTm, Be
4  else
5      $Key.PosCmd:Update (-5)
6  end
    
```

Figure E-27. Method content editor window

E.3.2.8.2 Delete Method

Right click on the **Method** you want to delete, select **Delete** or select **Method**, and then click the **delete key**.

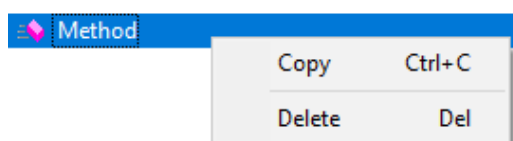


Figure E-28. Delete method

E.3.2.9 Variable node

E.3.2.9.1 Add Variable

Right click on the node you want to add a variable to, select **Add Node** or click **ctrl + N** to select **Variable**

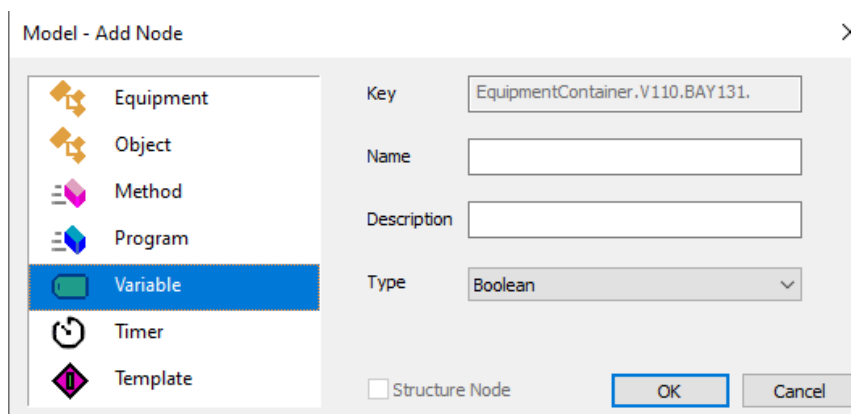


Figure E-29. Add a new variable menu

General	
Key	EquipmentContainer.V110.BAY131.Variable
Name	Variable
Description	
Type	Variable
Area	OCC_HNO
Excluded	☐

Variable	
Data Type	Boolean
Initial Value	False
Mapping Type	None
Alarm Type	None
☐ Historizing	☐ Retentive
☐ Latch	☐ Data Change Event
☐ Add to Trend View List	

Figure E-30. Variable node configuration

Fill in **Name**, **Description**, and select **Type** for the Variable you just created.

Selection of parameters:

- **Data type:** The data type of the variable.
- **Mapping Type:**
 - **None:** No data mapping.
 - **Point to Point:** Mapping point-to-point data.
 - **Point to Point (2-Source):** Point-to-point data mapping with 2 data sources.
 - **Function Block:** Mapping type data using function blocks.
 - **Script:** Mapping the type of data using scripts according to the user's wishes.

- **Initial Value:** The initial value of the variable.
- **Alarm Type:** Alarm type.
 - **Binary Alarm:** Corresponds to binary tags with two possible states:
 - **Alarm State:** The alarm state corresponds to the true/false value.
 - **Severity:** The priority of the alarm, ranging from 0 to 999, helps filter the alarm on the HMI.
 - **Message:** The content displays the alarm.
 - **Alarm Group:** A type of alarm group.
 - **Category:** Alarm grouping makes it easier to manage and track alarms.
 - **Sound ID:** Provides a warning feature for users by playing audio files when there is an alarm.

Binary Alarm

☒ Enable

Alarm State ☒ ON ☐ OFF

Severity

Inhibitor

Message

Alarm Group

Category

Sound ID

- **Discrete Alarm:** Corresponds to discrete tags with multiple possible statuses:
 - **Alarm State:** Enter the Value Set values, select State, Severity, Sound ID.
 - Other parameters are similar to binary alarms.

Discrete Alarm

☒ Enable

Alarm Group

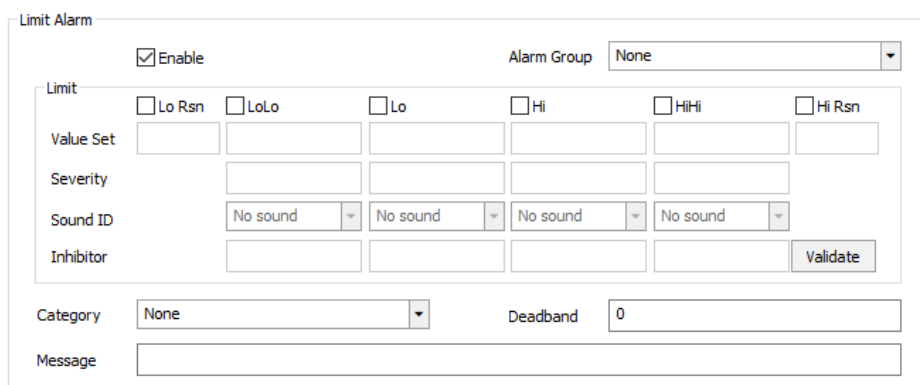
Alarm State

Value Set	State	Severity	Sound ID	Inhibitor	
0	Normal	0	No sound		☒
1	Alarm	400	BELL		☒
2	Alarm	400	BELL		☒

Category

Message

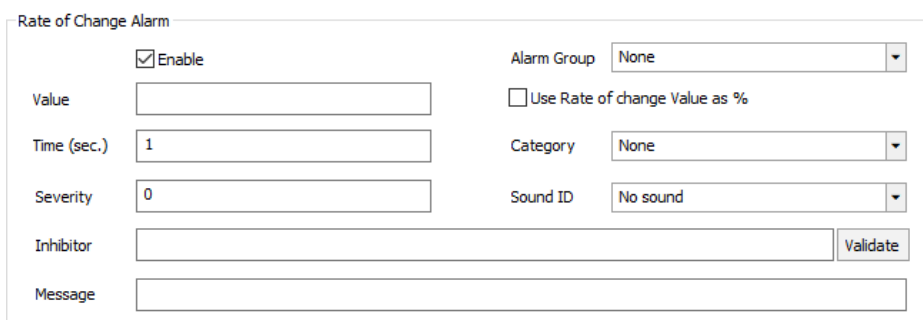
- **Limit Alarm:** Usually used for analog variables. The value of the variable is compared with the predetermined limit values, if the value exceeds that limit, the corresponding alarms will arise:
 - **Value Set:** The value of the corresponding alarm threshold.
 - Other parameters are similar to binary alarms.
 - **Deadband:** Fill in the number where the change in the value of the tag must be greater than this value to have the alarm state transition.



The 'Limit Alarm' form includes the following fields and controls:

- Enable:** A checked checkbox.
- Alarm Group:** A dropdown menu set to 'None'.
- Limit:** A section containing six checkboxes: 'Lo Rsn', 'LoLo', 'Lo', 'Hi', 'HiHi', and 'Hi Rsn'. Below these are corresponding input fields for 'Value Set', 'Severity', and 'Sound ID' (each with a 'No sound' dropdown). An 'Inhibitor' input field is also present.
- Category:** A dropdown menu set to 'None'.
- Deadband:** An input field containing the value '0'.
- Message:** A text input field.
- Validate:** A button located at the bottom right of the 'Limit' section.

- **Rate of Change Alarm:** Evaluate the rate at which the value of the tag changes to determine if an alarm has occurred:
 - **Rate of change Value:** The threshold set for the alarm, which can be used as a percentage instead of a value by checking Use Rate of change Value as %.
 - Other parameters are similar to binary alarms.
 - **Time (sec.):** The interval in seconds to calculate the rate of change.



The 'Rate of Change Alarm' form includes the following fields and controls:

- Enable:** A checked checkbox.
- Alarm Group:** A dropdown menu set to 'None'.
- Value:** An input field.
- Use Rate of change Value as %:** An unchecked checkbox.
- Time (sec.):** An input field containing the value '1'.
- Category:** A dropdown menu set to 'None'.
- Severity:** An input field containing the value '0'.
- Sound ID:** A dropdown menu set to 'No sound'.
- Inhibitor:** A text input field.
- Message:** A text input field.
- Validate:** A button located at the bottom right of the 'Inhibitor' field.

- **Historizing:** Storing palanquin data in HIS. For analog data, it is necessary to configure the parameters, the meaning of the parameters is seen in the HIS Server document.

History

Zero Deviation

☐ Step

Exception

Deviation

	Day	Hour	Min	Sec
Min. Time	0	0	0	0
Max. Time	0	0	1	0

Unexpected Change

Multiples of Deviation

Time Estimate (s)

Compression

Deviation

	Day	Hour	Min	Sec
Min. Time	0	0	0	0
Max. Time	0	1	0	0

- **Retentive:** The current value of the variable is backed up and can be restored to the variable in the event of an application restart.
- **Latch:** Applies to Boolean-style variables. When the variable changes the value from false to true.
- **Data Change Event:** Generates an event whenever the value of a variable changes.
- **Add to Trend View List:** Allows to graph the trend of the variable.
- **Data Source:** Data source.
- **Scale Factor:** Choose a scale scale

Press the ctrl + S key combination to save.

E.3.2.9.2 Delete Variable

Right-click on the **variable** you want to delete, select **Delete** or select **Variable** and then press the delete key.

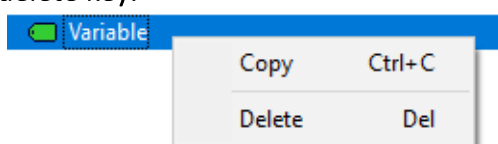
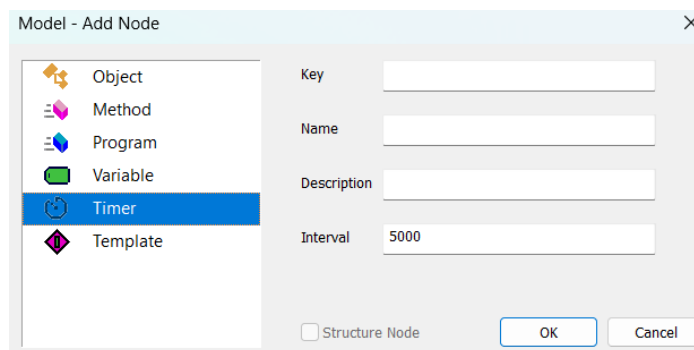


Figure E-31. Variable removal menu

E.3.2.10 Timer

From the node that wants to add new Timer, right-click and select **Add/Timer:**



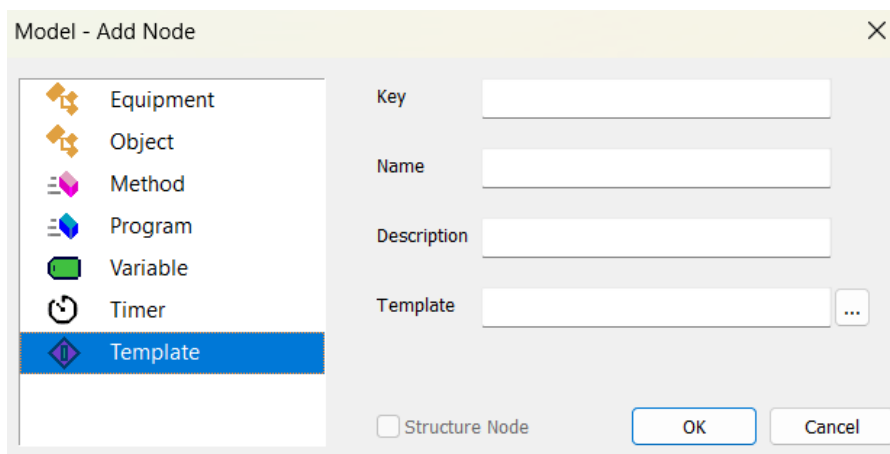
The dialog box 'Model - Add Node' has a left sidebar with icons for Object, Method, Program, Variable, Timer, and Template. The 'Timer' icon is selected. On the right, there are input fields for Key, Name, Description, and Interval (set to 5000). There is a checkbox for 'Structure Node' and 'OK'/'Cancel' buttons.

Fill out **Name**, **KeyName**, **Description** and Timer - Interval depending special purpose.

E.3.2.11 Node templates

E.3.2.11.1 Add a Template

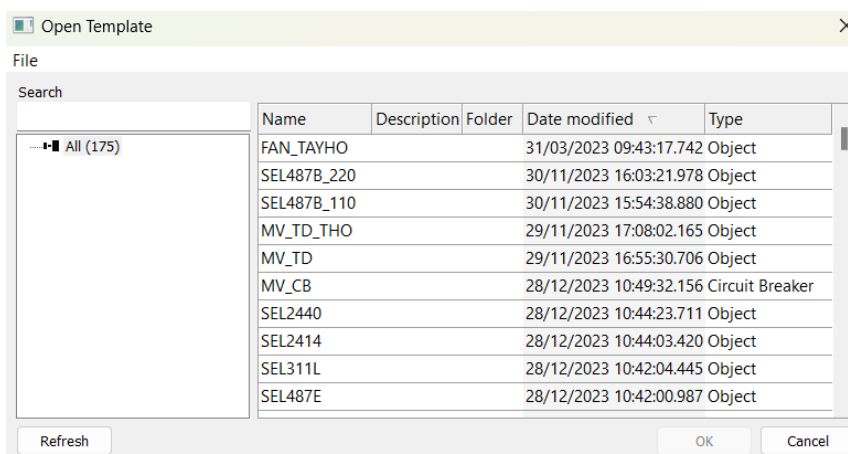
Right-click on the node you want to add a template to, select **Add Node** or click **ctrl + N** to select **Template**



The dialog box 'Model - Add Node' is shown with the 'Template' icon selected in the sidebar. The right side has input fields for Key, Name, Description, and Template (with a dropdown arrow). There is a checkbox for 'Structure Node' and 'OK'/'Cancel' buttons.

Figure E-32. Add a new node from the template menu

Fill in **Key**, **Name**, **Description**, select the template you want to add according to the Template list.



The 'Open Template' dialog box shows a file explorer view. On the left is a search bar and a list of files (175 items). On the right is a table of templates.

Name	Description	Folder	Date modified	Type
FAN_TAYHO			31/03/2023 09:43:17.742	Object
SEL487B_220			30/11/2023 16:03:21.978	Object
SEL487B_110			30/11/2023 15:54:38.880	Object
MV_TD_THO			29/11/2023 17:08:02.165	Object
MV_TD			29/11/2023 16:55:30.706	Object
MV_CB			28/12/2023 10:49:32.156	Circuit Breaker
SEL2440			28/12/2023 10:44:23.711	Object
SEL2414			28/12/2023 10:44:03.420	Object
SEL311L			28/12/2023 10:42:04.445	Object
SEL487E			28/12/2023 10:42:00.987	Object

Buttons: Refresh, OK, Cancel

Figure E-33. List of templates

General

Key

AC1

Type

Template

Name

AC1

Area

D01

Description

☐ Excluded

Template

Template Reference:

AC_MMxu

...

Input Tag

Input Text

Input Constant

Input Sound

Custom Data

No.	Input Id	Input Tag
1	MMxu1_APhsA	
2	MMxu1_APhsB	
3	MMxu1_APhsC	
4	MMxu1_totW	
5	MMxu1_totVAr	
6	MMxu1_S	
7	MMxu1_totPF	
8	MMxu1_PPVPhsAB	
9	MMxu1_PPVPhsBC	
10	MMxu1_PPVPhsCA	
11	MMxu1_PhVphsA	
12	MMxu1_PhVphsB	
13	MMxu1_PhVphsC	
14	MMxu1_Hz	
15	Online	

Figure E-34. Configuring a node from a template

Mapping for Import **Tags**.

Configure input parameters: **Input Text**, **Input Constant**, **Input Sound**, **Custom Data**.

Press the ctrl + S **key combination** to save.

E.3.2.11.2 Delete Template

If the template is **Extracting**, Right Click the template and select **Compress**.

Right-click on the **Template** you want to delete, select **Delete** or select **Template** and then click **the delete key**

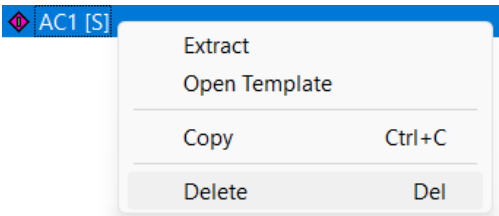


Figure E-35. Delete a build node from a template

E.3.2.12 Exclude nodes in the datamodel

Exclude node is a feature that excludes that node in the dataserver's calculations.

Click on the node you want to exclude, select **Excluded**.

For example, as the Node BAY431 figure is configured for Tagging:

On the HMI's datamodel, E1_16.V22 variables will be automatically generated. BAY431.Tagging.

E.3.2.14 Configuring Alarm category general in the datamodel

Alarm category general is a feature that counts the number of datapoints with the same alarm category that are active in the node.

Click on the node that needs to configure alarm category general, select **Alarm category general**.

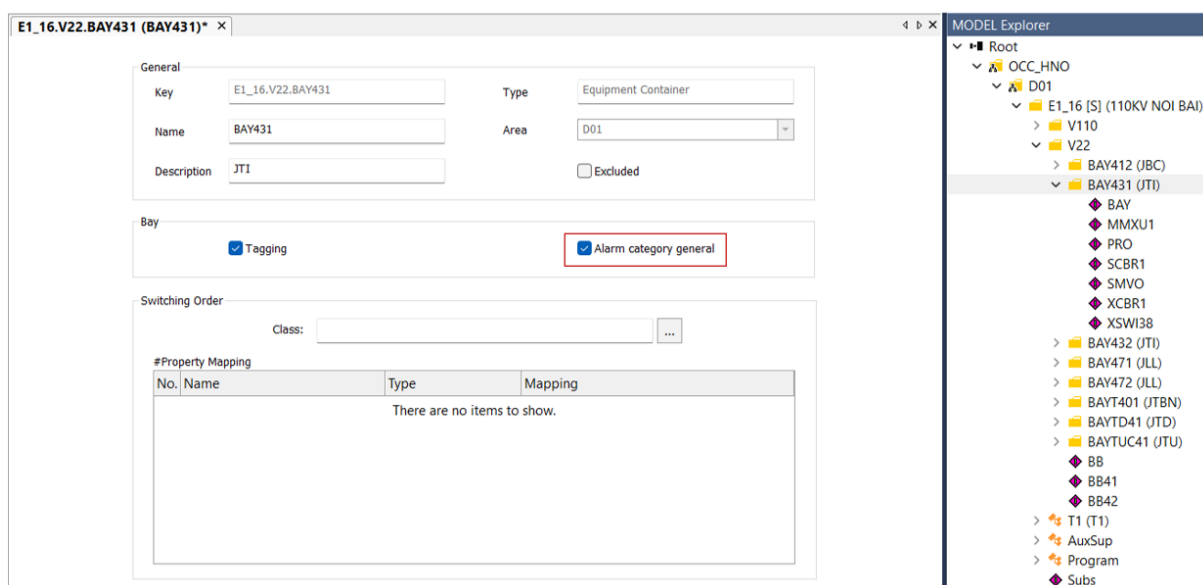
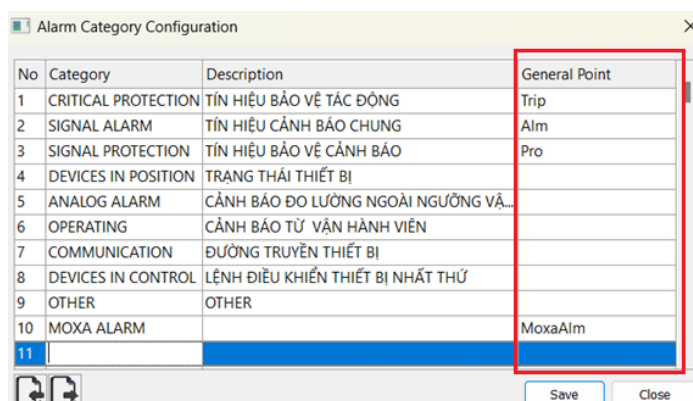


Figure E-38. Alarm category general configuration

Nodes configured with alarm category general, on Grid Studio's datamodel (HMI) will automatically generate the "Key.General Point".

In which:

- **Key:** is the key of the node configured with Alarm category general.
- **General Point:** is the General Point name configured in the Alarm Category.



For example, as shown in Node BAY431 configured alarm category general:

- On the HMI's datamodel, E1_16.V22 variables will be automatically generated. BAY431. Trip, E1_16.V22. BAY431. Alm, E1_16.V22. BAY431. Pro, E1_16.V22. BAY431. MoxaAlm.
- The value of the variable E1_16.V22. BAY431. Trip: this variable is equal to the total number of datapoints in the BAY431 node with the same category type "CRITICAL PROTECTION" that is in the ACTIVE state.
- The value of the variable E1_16.V22. BAY431. Alm: this variable is equal to the total number of datapoints in the BAY431 node with the same category type "SIGNAL ALARM" that is in the ACTIVE state.
- The value of the variable E1_16.V22. BAY431. Pro: this variable has a value equal to the total number of datapoints in the BAY431 node that shares the same category type "SIGNAL PROTECTION" that is in the ACTIVE state.
- The value of the variable E1_16.V22. BAY431. MoxaAlm: this variable has a value equal to the total number of datapoints in the BAY431 node with the same category type "MOXA ALARM" that is in the ACTIVE state.

E.3.2.15 Configuring Switching Orders in the datamodel

E.3.3. Data Mapping

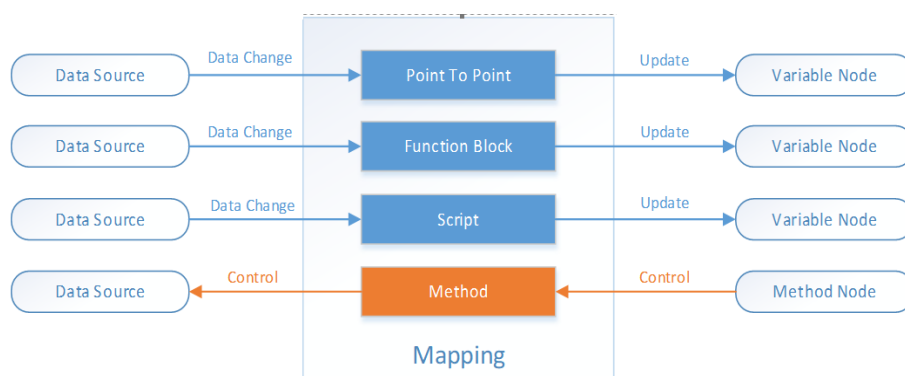


Figure E-39. Data mapping

Data mapping is a method of data exchange:

- Between the **Variable node** on the DataModel side and the data point on the FEP module.
- Or between **Variable nodes**.

Mapping Types:

- **Point to point:** Output updates directly from Input.
- **Point to point (2- Sources):** Directly updated output from 1 of 2 Input Data Sources
- **Function Block:** Output updates from Input through compute blocks.
- **Script:** Write a script to update the Output according to the Input.
- **Method:** Method for control.

- **Expression:** Output is the output value of an expression

E.3.3.1 Configure Point to Point mapping

When configuring the **Variable**, click the ▼ icon and select **Point to Point**.

As for the **Data Source**, we can fill it in manually by copying the key name from **FEP** or copying the key from **MODEL**.

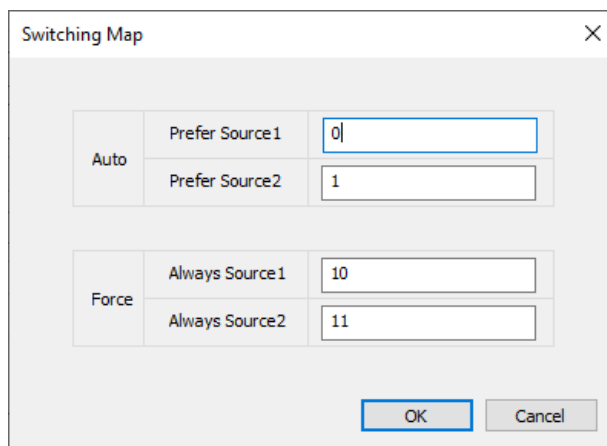
E.3.3.2 Point to Point (2-Source) mapping configuration

When configuring **Variable**, click the icon ▼ and select **Point to Point**.

Data Source 1, **Data Source 2** section, we can manually fill in or drag drop from the **FEP** (Format FEP::...) or **MODEL** (Format MODEL::...).



- ❖ If **Switching** is not selected on the mapping panel, the DataServer automatically selects the source 1 for the data, the user cannot change the desired data source.
- ❖ If **Switching** is selected, the user can use the variable (**switching signal**) in the **Switching** box to change the data source selection mode, the Switch value is configured at the Switching Map (**MODEL->Configuration->Switching Map**), this value is used for the whole project.
- ❖ If **Switching** is selected but the switching box is blank, the DataServer automatically selects the source 1 for the data, too.
- ❖ If **Switching** is selected but the mapping configuration of a point leaves "Source 1" or "Source 2" blank, then this mapping becomes a normal Point to Point mapping (only 1 signal source), the program will always select that data source and the variable "Switch signal" changes does not affect to that mapping.
- ❖ **Switching signal** must have good quality while changing value, if it change to bad quality, the DataServer will remain state of mapping.
- ❖ If **Switching signal** value changes outside the configuration table "Switching Map", the DataServer will remain state of mapping.



The image shows a 'Switching Map' dialog box with a close button (X) in the top right corner. It contains two main sections: 'Auto' and 'Force'. The 'Auto' section has two input fields: 'Prefer Source1' with the value '0' and 'Prefer Source2' with the value '1'. The 'Force' section has two input fields: 'Always Source1' with the value '10' and 'Always Source2' with the value '11'. At the bottom right, there are 'OK' and 'Cancel' buttons.

Figure E-40. Switching map

Switching Map has two options (**Auto, Force**)

E.3.3.2.1 Auto Switching Map

When the "**Switch signal**" (Switch) variable value is set in this range ($0 \leq \text{Switch} \leq 1$ via **Error! Reference source not found.**), the program (DataServer) will automatically check and choose mapping to the data source with good quality out of two data sources.

In case both data sources have the same quality (both Good or Bad), the priority of mapping to which data source will depend on the "Switch signal" value, if the value is "Prefer Source 1" (Switch = 0 via **Error! Reference source not found.**) then the program will select data source 1 and if the value is "Prefer Source 2" (Switch = 1 via **Error! Reference source not found.**), then the program will select data source 2.

The principle is that DataServer will check the quality of the signal sources first, then consider the variable value of "**Switch signal**" as "Prefer Source 1" or "Prefer Source 2" when both data sources have the same quality.

E.3.3.2.2 Force Switching Map

When the variable value "**Switch signal**" is set in this range, the mapping to which data source will be fixed. With the variable value "**Switch signal**" being:

- Always Source 1: (Switch = 10 via **Error! Reference source not found.**) the program will choose data source 1, whether the quality is Bad or Good.
- Always Source 2: (Switch = 11 via **Error! Reference source not found.**) the program will choose data source 2, whether the quality is Bad or Good.

The principle is that the DataServer will check the value of the variable "Switch signal" and map it to the corresponding data source, and the quality will not be considered in this mode

E.3.3.3 Configure Function Block mapping

When configuring the **Variable**, click the ▼ icon and select **Function Block**.

In the **Mapping section**, click **Edit** to open the **Mapping window**.

Click **Create**, the interface window allows the user to use compute blocks to calculate the result for the output variable.

Use drag and drop variables from FEP or MODEL to create input variables, drag and drop to get compute blocks, hold down the left mouse to connect the output to the input between the compute blocks.

Press the **ctrl + S** key combination to save.

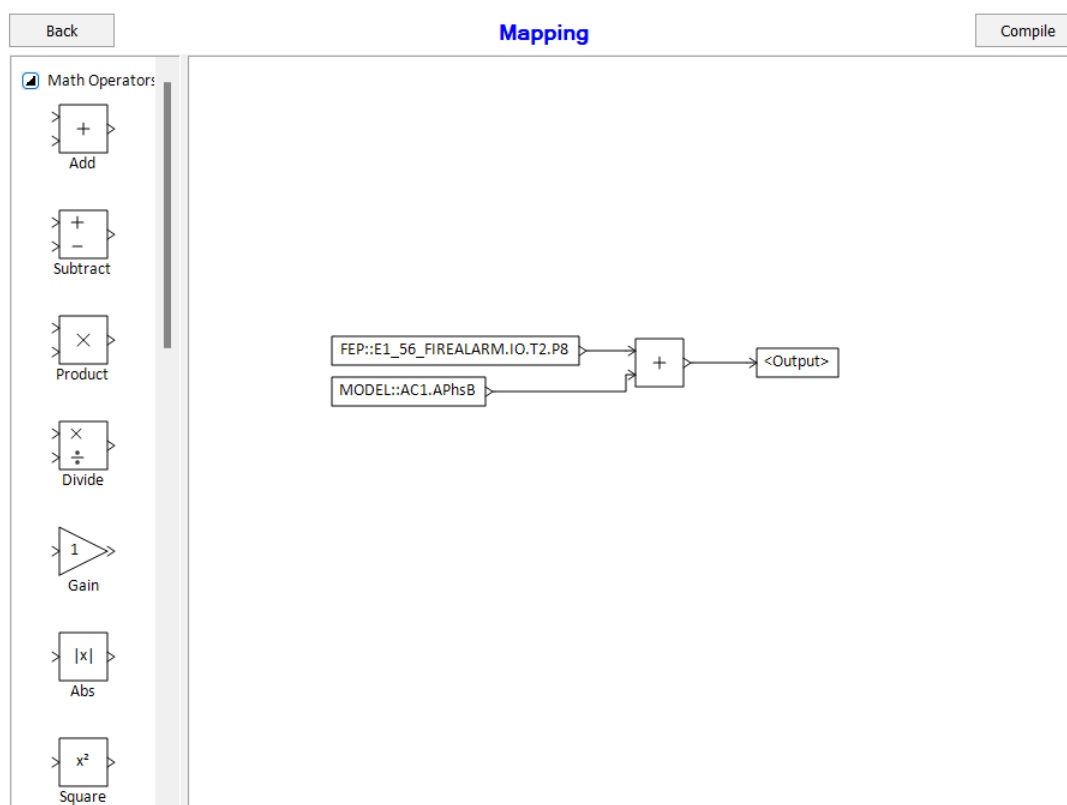


Figure E-41. Using Compute Blocks

E.3.3.4 Configure Script mapping

When configuring the **Variable**, click the ▼ icon and select **Script**.

In the Mapping **section**, click **Edit** to open the script editing window.

The script structure is similar to building a **program**.

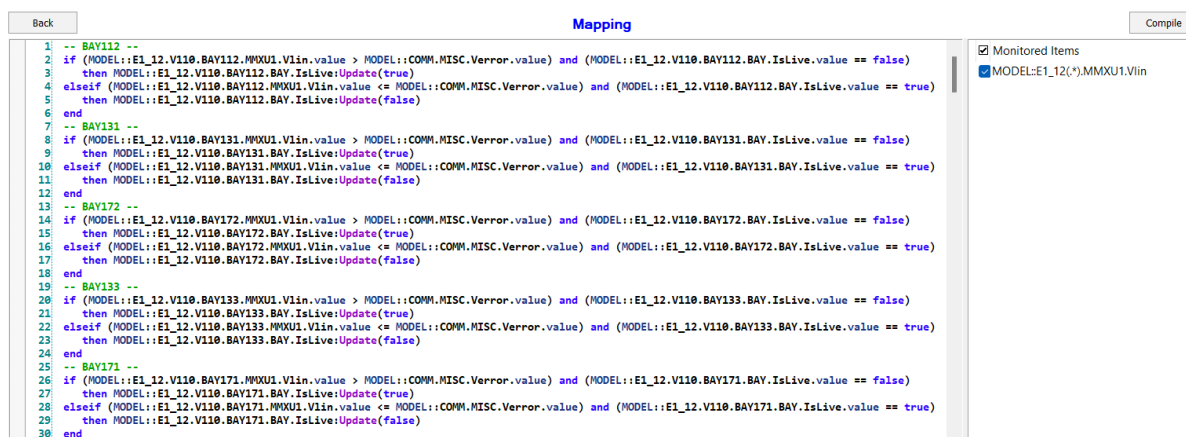


Figure E-42. Script-style data mapping

E.3.3.5 Method mapping

Use for **Import Tags of Template** or mapping for control variables on IEC101, IEC104 channels.

Used for mapping from Model to target Tags on FEP Slave Channel (Slave IEC104, Slave IEC101, Slave Modbus...) and do the control by Method Mapping from Method point in FEP Slave channel

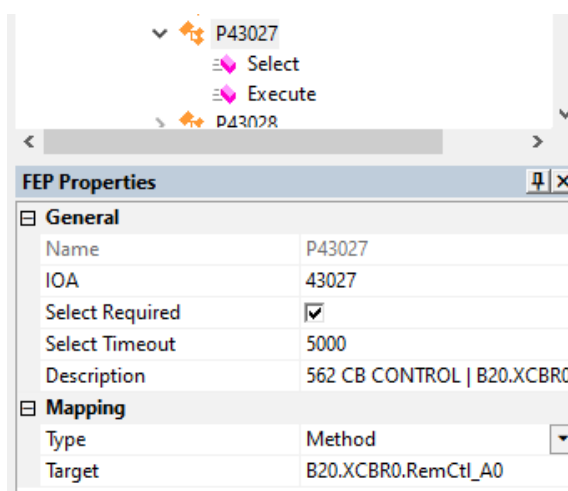
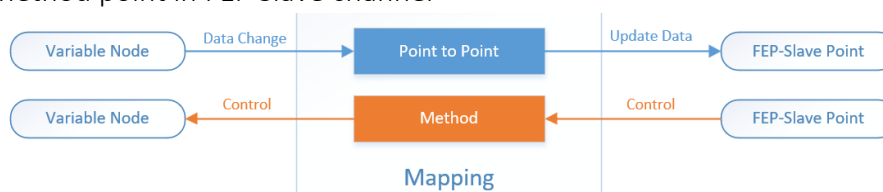


Figure E-43. Method mapping for control points

E.3.3.6 Expression mapping

Used for variable mapping for points on channels 104, 101. The variables in the calculation expression are in the [].

FEP Properties	
General	
Name	P1
IOA	1
Description	171 SF6 ALARM
Spontaneous Update	With 7 byte timestamp
Mapping	
Type	Expression
Target	[SF61] or [SF62]

E.3.4. Template

E.3.4.1 Template Concept

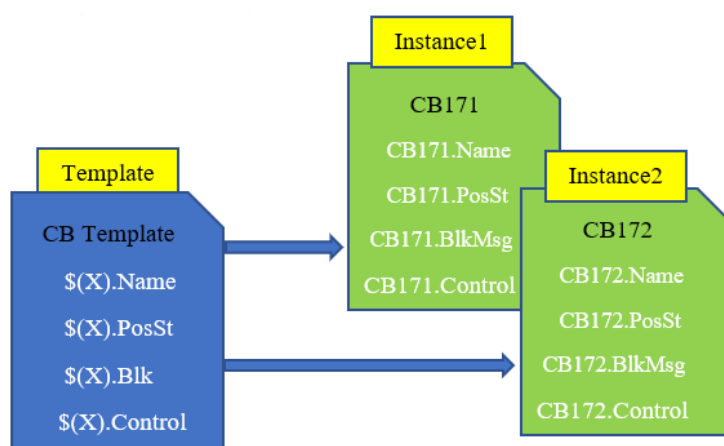


Figure E-44. Build different objects from the template

A **template** is a template object that has the right characteristics pre-designed by the user and is reusable.

The benefits of using templates are uniformity, efficiency when calibrating, saving time building shared objects from templates

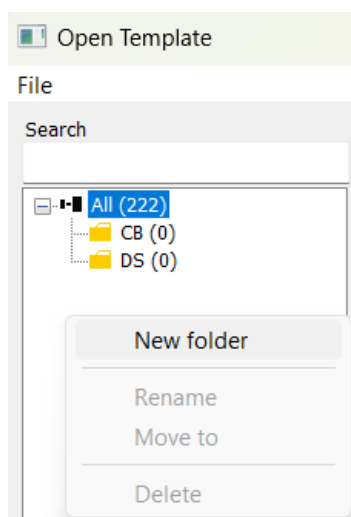
The objects that should be built Template for reuse are: cutters, isolation knives, exposed bays, etc.

Templates can also contain other templates

E.3.4.2 Build a Template

E.3.4.2.1 Add a new folder manager template

From the menu bar, select **Model >> Template Builder**, Right Click on the empty box to select **New Folder**:



In addition, it is also possible to add folders, **rename**, **move to**, **delete** folders by Right Click to the folder, select the corresponding actions.

E.3.4.2.2 Add a new template

There are 2 ways to add a new template: manually and import from file.

➤ **Handicraft**

From the menu bar, select **Model >> Template Builder >> File >> New Template**:

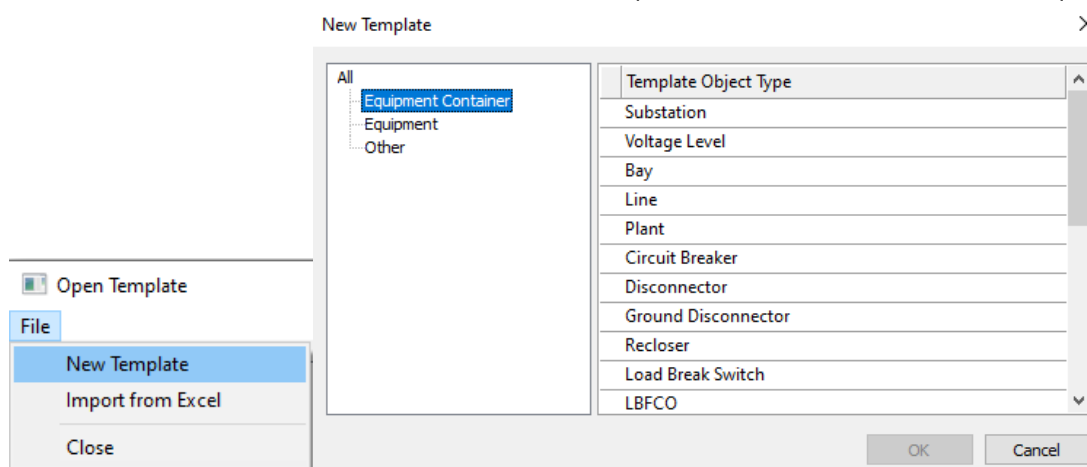


Figure E-45. Manual template builder menu

Select the **Template Object Type** for the newly created Template and click **OK**:

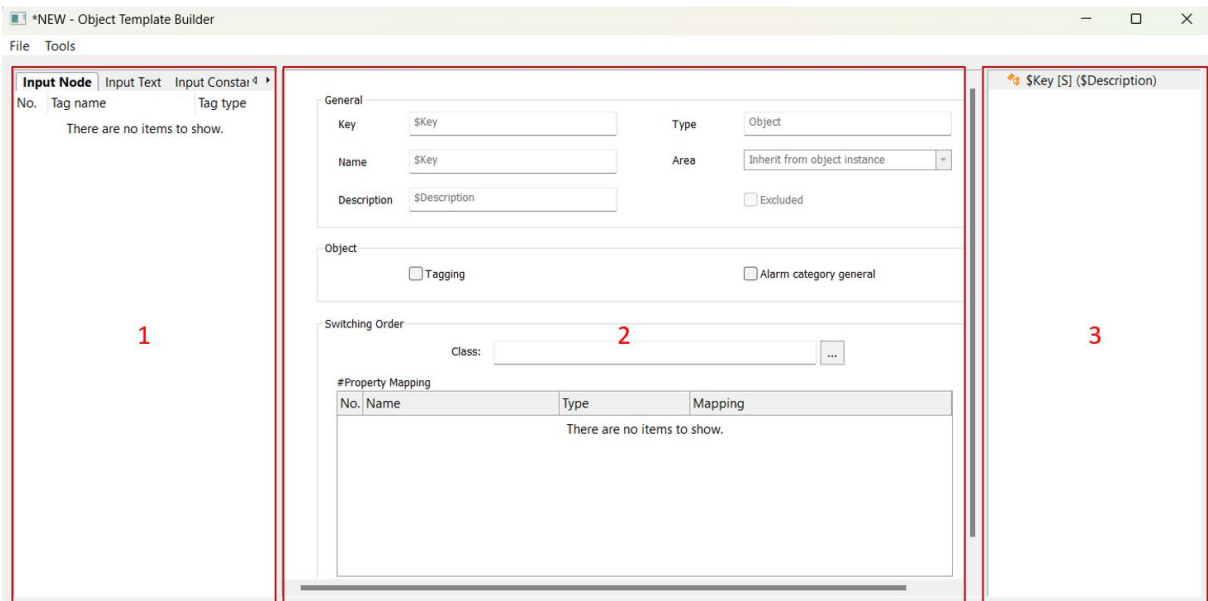
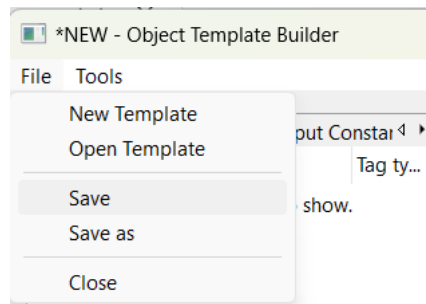


Figure E-46. Interface Object Template builder

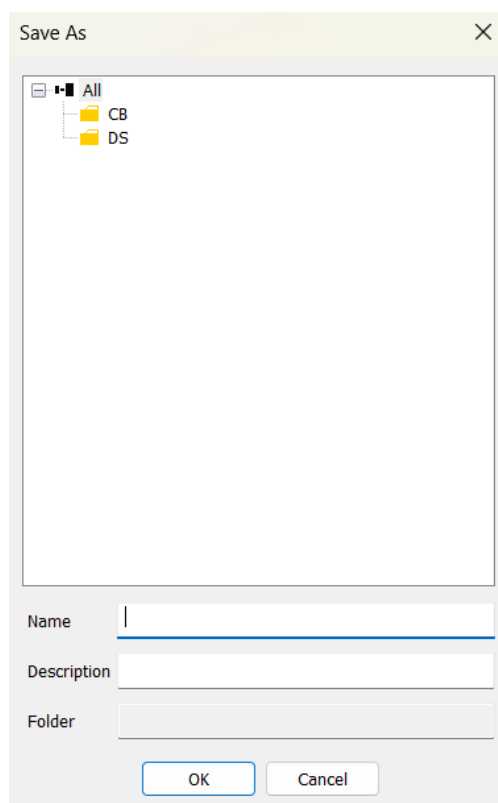
The Object Template Builder **interface** is divided into 3 areas:

- **(1): Contains** the input declarations of the template.
- **(2):** Configuration for the data node.
- **(3):** Contains variables, methods, programs, and other templates designed for the template.

After configuring the template, select **File >> Save**.

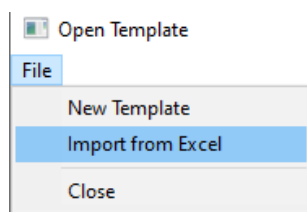


Fill in the **Name**, **Description**, select the **Folder** containing the template and click **OK** to save it.

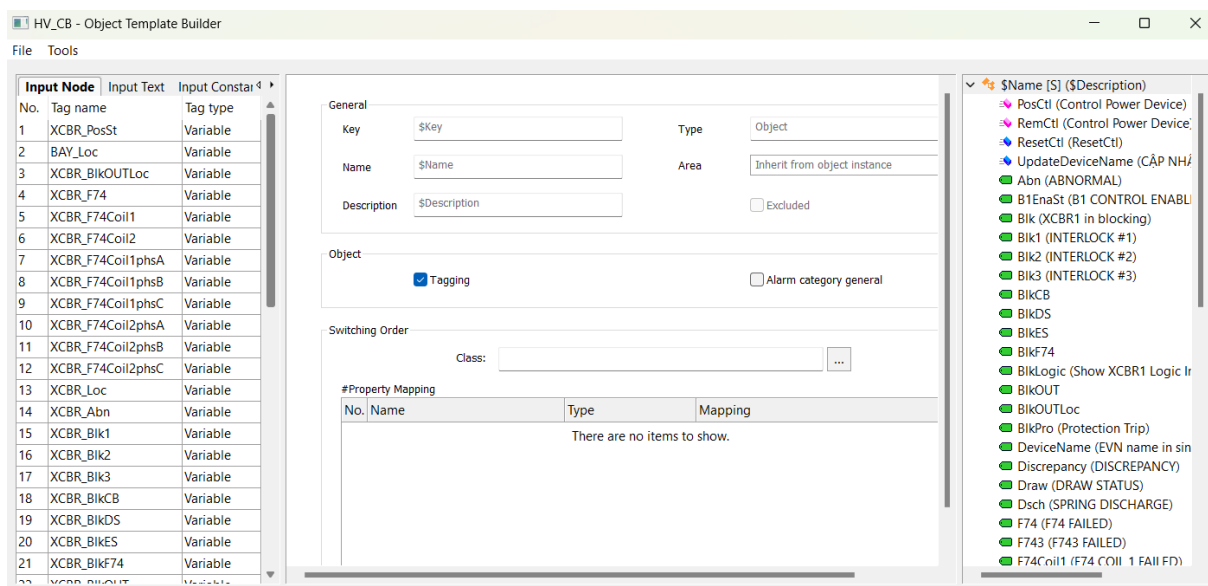


➤ Import from file

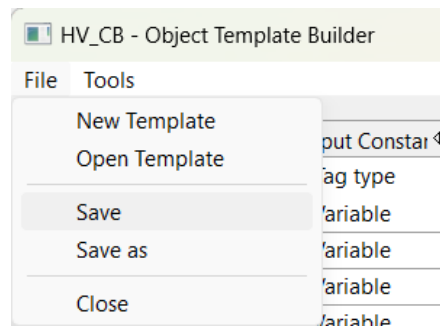
From the menu bar, select **Model >> Template Builder >> File >> Import from Excel**:



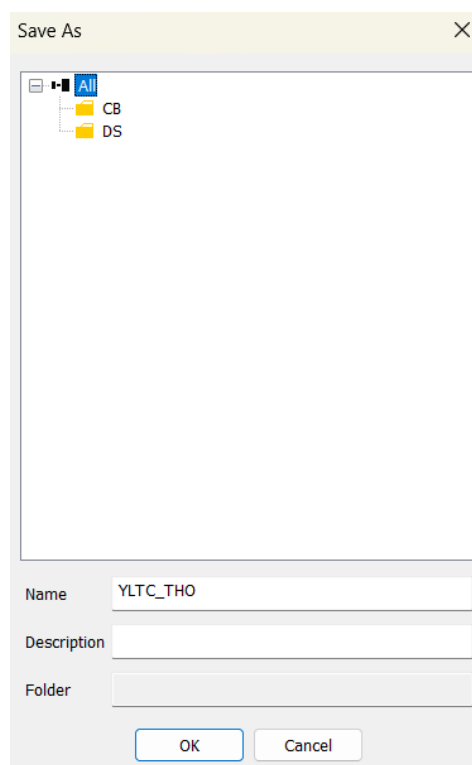
Select the pre-built template file, select Open:



Select **File >> Save**.



Fill in the **Name**, **Description**, select the **Folder** containing the template and click **OK** to save it.



E.3.4.2.3 Configuring Data for Templates

Configuring data nodes for a template is similar to configuring data for a DataModel.

The data nodes in the template are more limited than the data tree on the DataModel. Data nodes can be **Object**, **Method**, **Program**, **Variable**, **Timer**, and other **Templates**.

E.3.4.2.4 Declare Input Variables

In the **Input Node tab**, right-click and select **Insert new (ctrl+N)** to add a new input variable to the template:

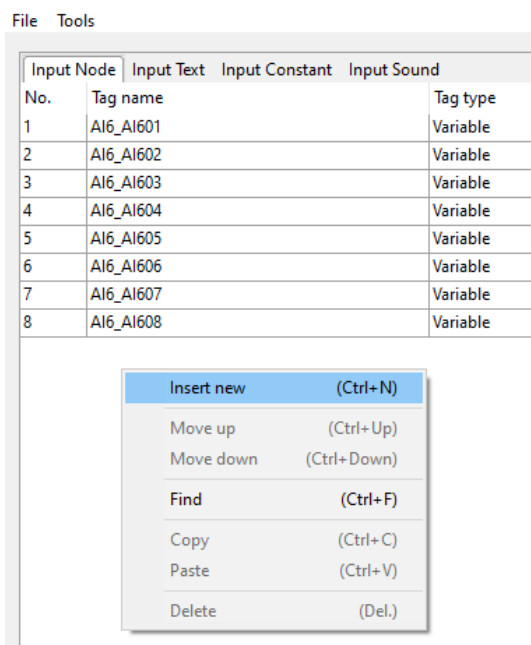


Figure E-47. Variable declaration menu

Fill in the **Tag name** and select the Tag type data type for the variable:

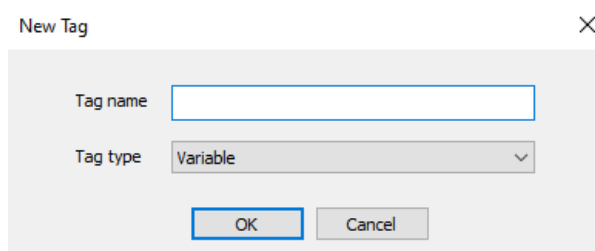


Figure E-48. Variable Configuration

Perform **Copy/Paste/Delete** an input variable by selecting the corresponding feature in the dialog box that appears when right-clicking.

To move a variable up/down in the list, click on the variable, hold down the Ctrl key, and press the up/down navigation key.

Declaring **Input Text**, **Input Constant**, **Input Sound** performs the same as **Input Node**.

➤ **Custom Data Schema**: declares specific scenarios for different purposes.

E.3.4.3 Other actions

E.3.4.3.1 Open Template

From the menu bar, select **Model >> Template Builder**, the list of templates will open, Double clicks on the template you want to open:

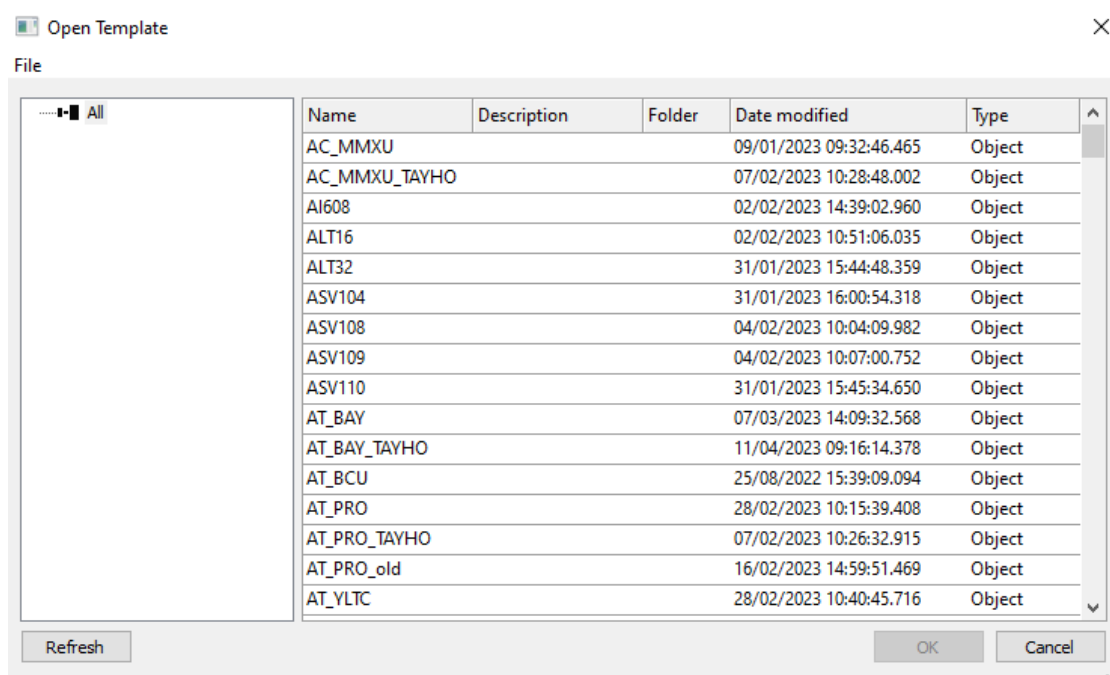
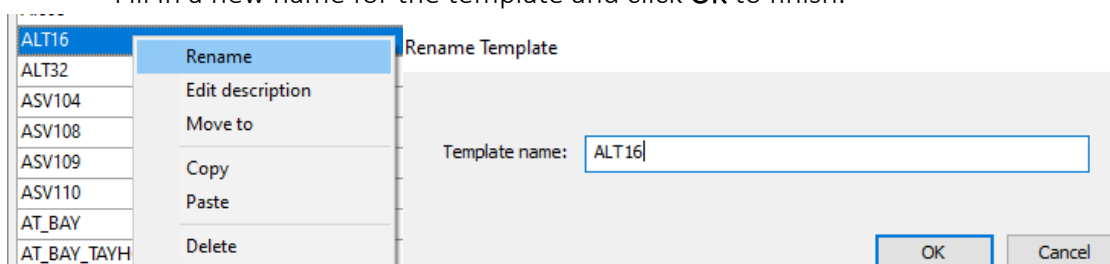


Figure E-49. Menu to open template

E.3.4.3.2 Rename Template

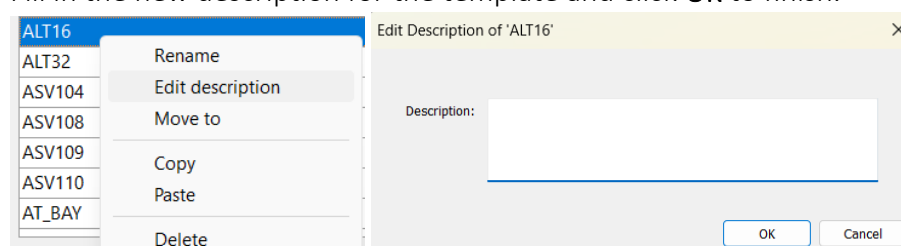
Right-click on an existing template in the **Open Template** screen and select **Rename**. Fill in a new name for the template and click **OK** to finish.



E.3.4.3.3 Modify the description of the Template

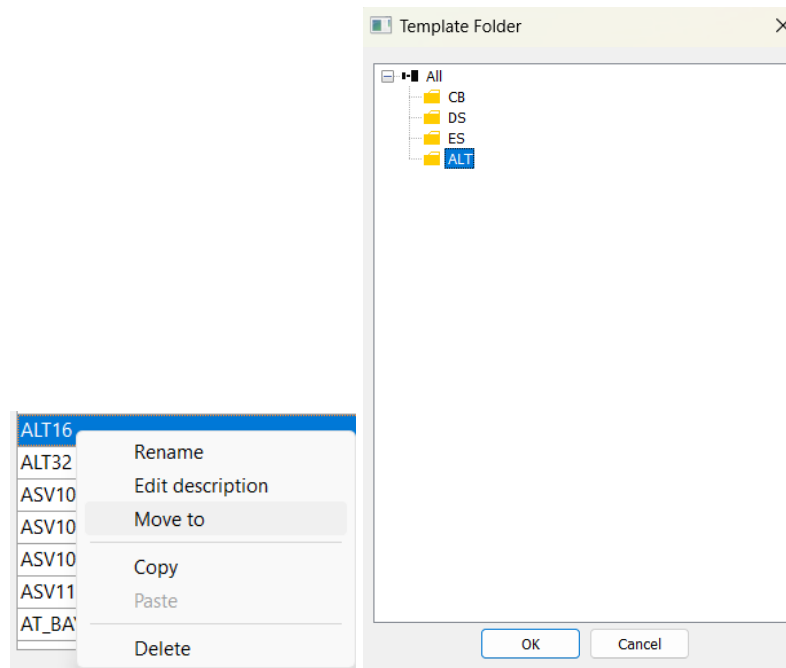
Right-click on an existing template in the **Open Template** screen and select **Edit description**.

Fill in the new description for the template and click **OK** to finish.



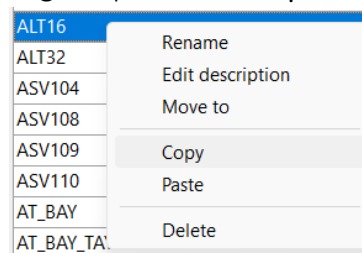
E.3.4.3.4 Move the template to another management location

Right-click on an existing template in the **Open Template** screen and select **Move to**. Select the template management folder and click **OK** to finish.

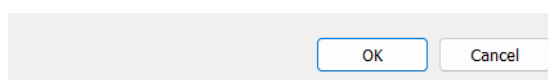
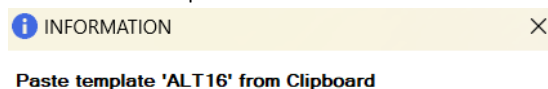


E.3.4.3.5 Copy/Paste Template

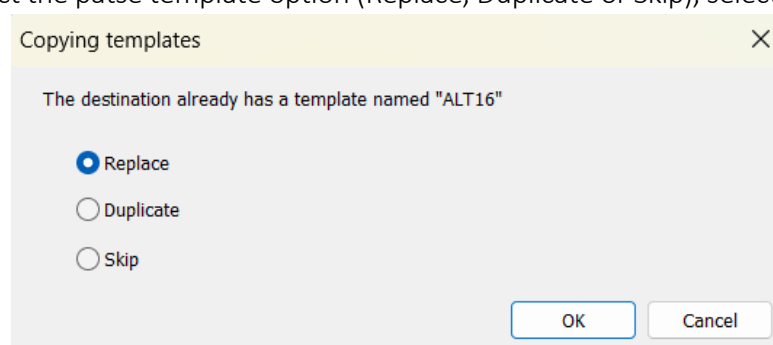
Right-click on an existing template in the **Open Template screen** and select **Copy**.



Right Click select Paste and press **OK** to continue.



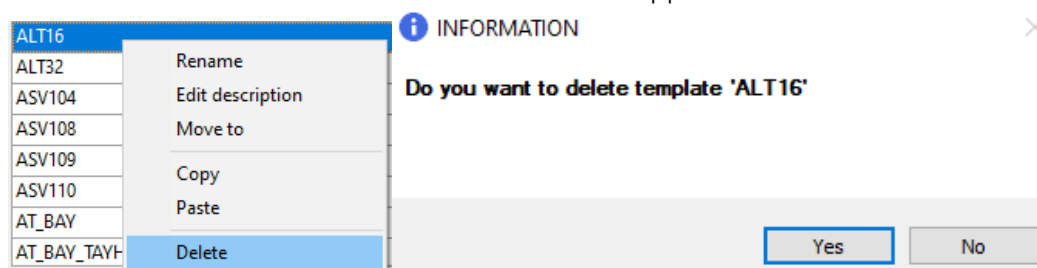
Select the patse template option (Replace, Duplicate or Skip), select **OK** to finish.



E.3.4.3.6 Delete an existing template

Right-click on an existing template in the **Open Template screen** and select **Delete**.

Select **Yes** in the confirmation window that appears.



Consider before deleting templates, instances of deleted templates will cause errors during the project compilation process.

E.3.4.3.7 Import Template

Open the Template and select **Tools >> Import**:

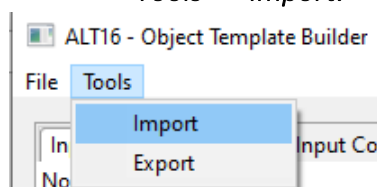


Figure E-50. Template import menu

Select the template structure file you want to import and click Open :

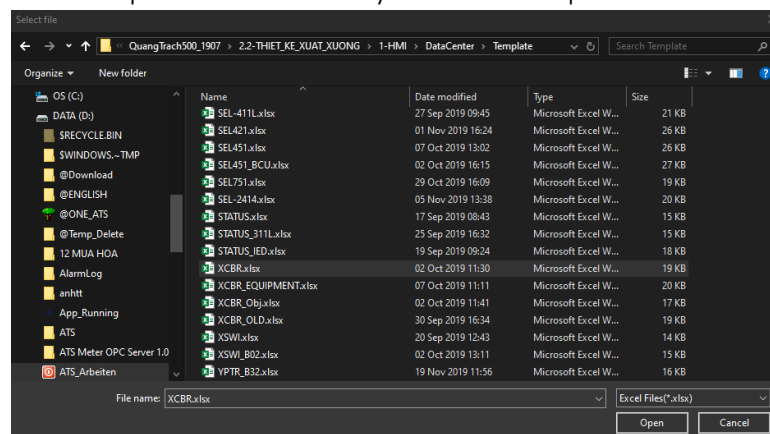


Figure E-51. Select a template file to import

E.3.4.3.8 Export Template

Open the Template and select **Tools >> Export**:

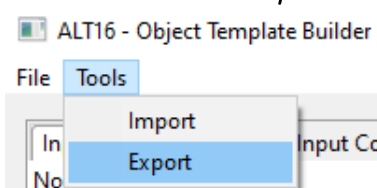


Figure E-52. Menu export template

Select the folder where you want to save the template file and click **Save**:

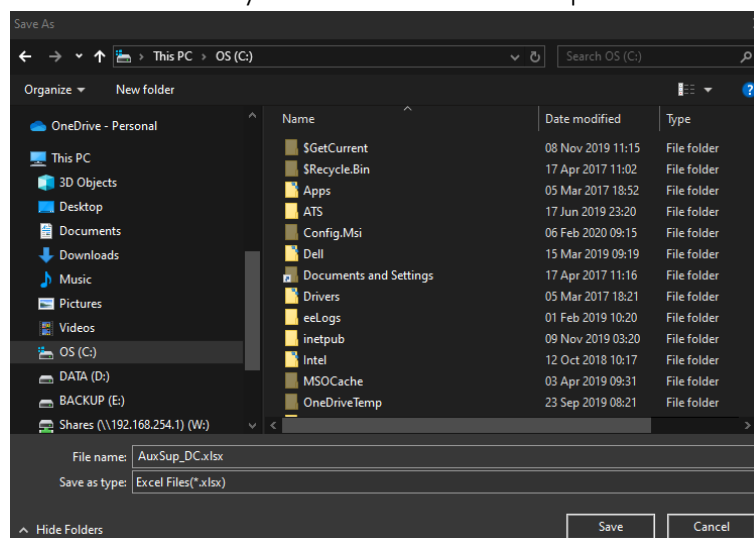


Figure E-53. Choose where to save your template

E.3.5. Variables Node

E.3.5.1 Naming rules

Do not use special characters or digits for Name/KeyName.

E.3.5.2 Classify

E.3.5.2.1 By range

The **Local** variable: is created and used within the scope of the application.

Global variables: have scope for use within the application and external applications.

E.3.5.2.2 By data type

Includes the data types as described in the OneATS Grid Studio module. There are built-in data types and user-defined data types.

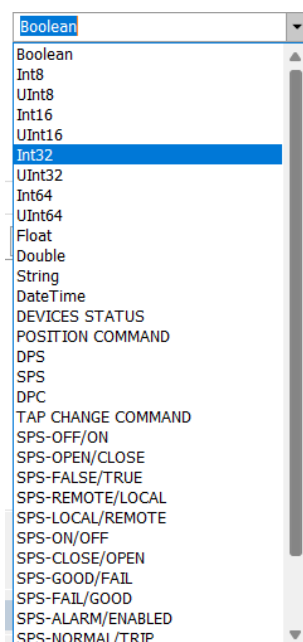


Figure E-54 Variable Data type

Digital Set: a user-defined data type.

User can self-define Data type in **Digital Set** tool. Digital Set is a set of state, which user want to self-define. Digital Set are made up of a set of arbitrary state-defined integer values.

Digital Set will also use in HIS.

Refer Item E.3.1.7 to configure **Digital Set**

E.3.5.2.3 Initial Value

Use the automatic initial value or choose a new initial value from **Initial Value** box.

E.3.5.3 Properties of variables

E.3.5.3.1 Name

The name displayed for the variable.

E.3.5.3.2 Key

Names are used in scripts and assignments.

E.3.5.3.3 Retentive

The current value of the variable is backed up and can be restored to the variable in the event of an application restart.

E.3.5.3.4 Latch

Applies to Boolean **type variables**. When a variable changes its value from false to true, it will have a value of true until it is reset latch.

E.3.5.3.5 Historizing

Save the variable's data to HIS.

E.3.5.3.6 Add to Trend View List

Allows to graph the trend of the variable.

E.3.5.3.7 Data Change Event

An event will be fired every time the value of the variable changes.

E.3.6. Alarm Configuration

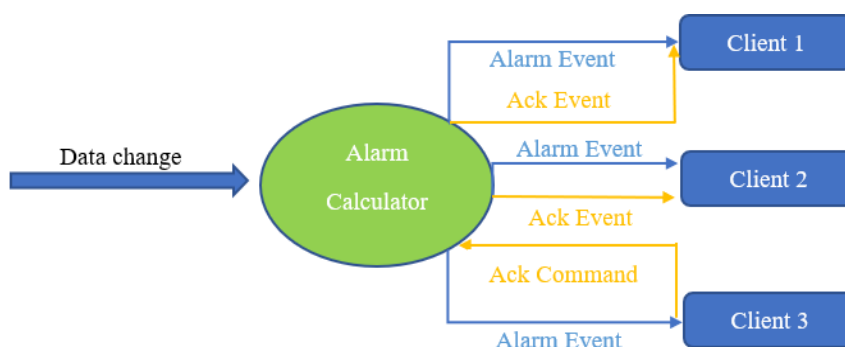


Figure E-55. Principle of alarm generation

E.3.6.1 Classify

E.3.6.1.1 "None"

"None" means this variable has no alarm configuration.

E.3.6.1.2 Binary alarm

Binary alarms correspond to binary tags with two possible states. When creating a new binary tag, the user can configure the alarm state corresponding to the true or false value.

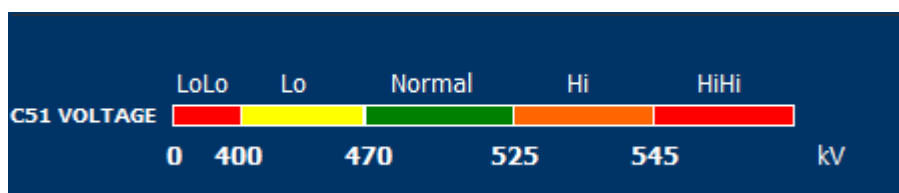
E.3.6.1.3 Discrete alarm

Corresponding discrete tags with multiple possible states, these states are represented by discrete values.

E.3.6.1.4 Limit alarm

Commonly used for analog tags. The current value of the tag is compared to one or more predetermined limits. If the value exceeds that limit, corresponding alarms will arise.

Users can configure limits and serverity for alarms.

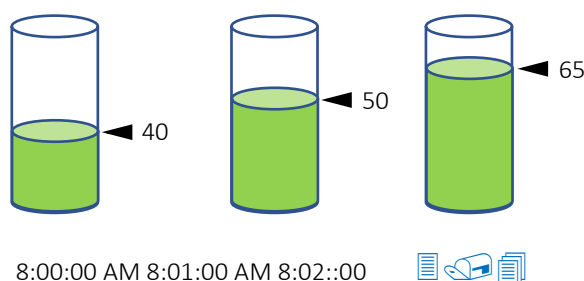


E.3.6.1.5 Rate of change alarm

Evaluate the rate at which the value of the tag changes to decide if an alarm is raised. Common examples are sudden loss of capacity, load layoffs.

The current value is compared to the previous value over a predetermined period of time to evaluate the rate of change. If the speed change exceeds a certain set limit, an alarm is raised.

The set limit can be a value or a percentage.



E.3.6.2 Attributes related to alarm

E.3.6.2.1 Severity

The parameter evaluates the priority (criticality) for an alarm. For example, a line breakdown that leads to turning on the cutter will be more important with a warning to open and close the cabinet door, which requires immediate attention from the operator.

Severity has a range from 0 to 999, with 999 being the highest priority.

Severity helps filter the alarms displayed on the HMI according to the user's wishes.

E.3.6.2.2 Alarm Category/group

Alarm grouping makes it easier to monitor and manage alarms.

There are many ways to group alarms such as by area, by device, by priority, etc.

Alarm grouping helps to filter the alarms displayed on the HMI according to the user's wishes.

When a new variable is created, it defaults to Category/Group as **None**.

E.3.6.2.3 Alarm sound

Provides a warning feature to users by playing audio files when there is an alarm.

The audio files that are usually associated with **the corresponding Severity** help the user immediately recognize the severity of the alarm.

E.3.6.3 Alarm Configuration

Use the **OneATS DataEditor** tool to open the project to configure the alarm. When a new tag is created, or tags already exist, the user has the customization to configure the alarm for that tag.

E.3.6.3.1 Binary Alarm Configuration

Reference [E.3.2.9 Variable node](#)

E.3.6.3.2 Discrete Alarm Configuration

Reference [E.3.2.9 Variable node](#)

E.3.6.3.3 Limit Alarm Configuration

Reference [E.3.2.9 Variable node](#)

Variables configured with Alarm Limit, on Grid Studio's datamodel (HMI) will automatically generate the "Key" variables. `_AlmLim`, `"Key. _AlmLim.Enabled"`, `"Key. _AlmLim.LoLo"`, `"Key. _AlmLim.Lo"`, `"Key. _AlmLim.Hi"`, `"Key. _AlmLim.HiHi"`, `"Key. _AlmLim.State"`.

In which:

- **Key:** is the Key of the variable configured Alarm Limit.

E.3.6.3.4 Rate of change Alarm Configuration

Reference [E.3.2.9 Variable node](#)

E.3.7. Import DataModel

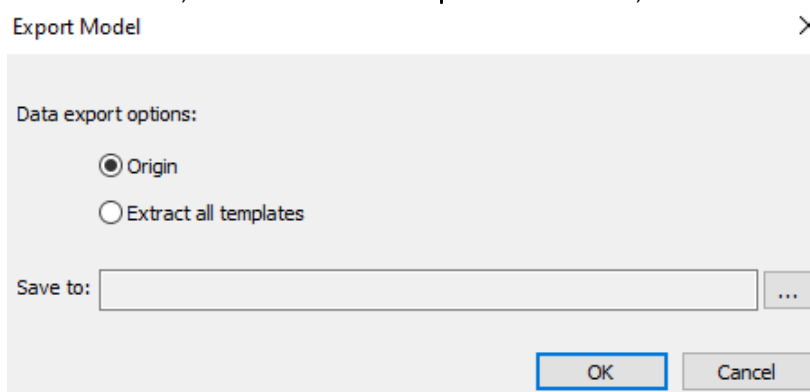
It is possible to import part or all of the DataModel. On the menu bar, select **Model >> Import from Excel**, select the prepared model file.

E.3.8. Export DataModel

It is possible to export all or part of the DataModel.

E.3.8.1 Export the entire DataModel

On the menu bar, select **Model >> Export from Excel**,

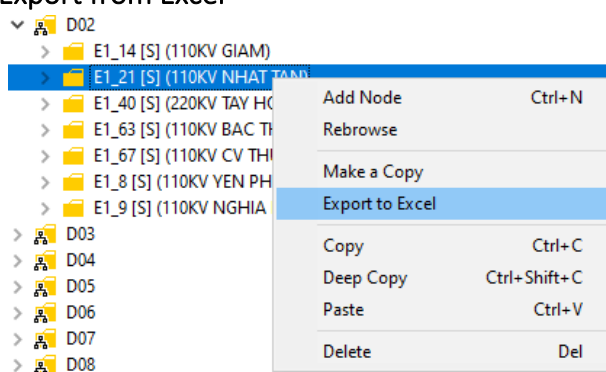


In which:

- **Origin:** DataModel will remain the same templates (recommended)
- **Extract all templates:** DataModel will break the templates
- **Save to:** where the DataModel file is stored

E.3.8.2 Export a part of the DataModel

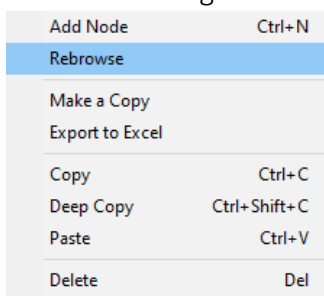
On the **MODEL Explorer window**, Right Click on the DataModel you want to export and select **Export from Excel**



The configuration parameters are similar to export the entire DataModel.

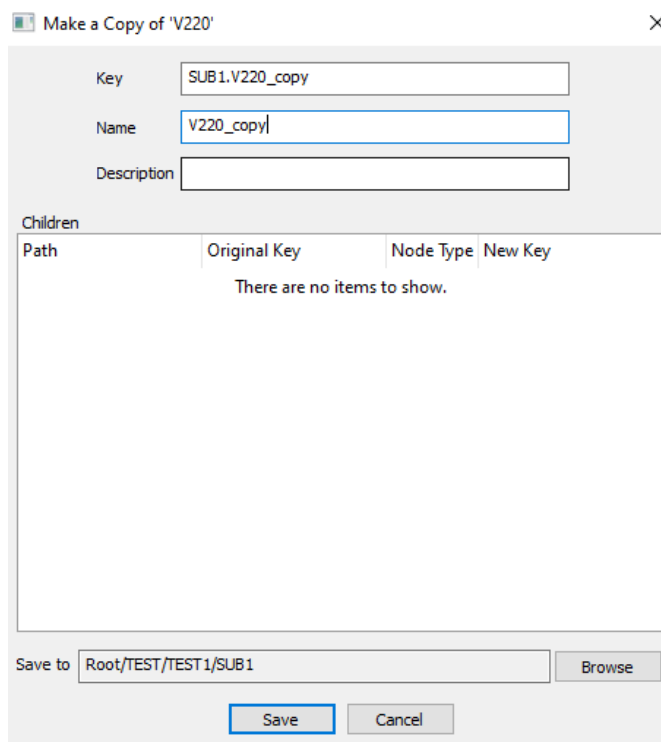
E.3.9. Some things to do when working with a DataModel

Some other things to do when working with a DataModel:



E.3.9.1 Make a Copy

Make a Copy is the operation of copying all the data in a node.



Make a Copy of 'V220'

Key: SUB1.V220_copy

Name: V220_copy

Description:

Children

Path	Original Key	Node Type	New Key
There are no items to show.			

Save to: Root/TEST/TEST1/SUB1

Browse

Save Cancel

Figure E-56. Make a Copy Interface on DataModel

In which:

- **Save to:** Where the node is saved
- **Name:** Node name

E.3.9.2 Copy

Copy is an operation that only copies the node name, does not copy the data in the node, especially for templates, the copy operation will copy all the data of the template.

The user selects the node to be copied and Right-Click to select **Copy** or press **Ctrl + C**.

The user selects the location to save and Right-Click selects **Paste** or presses the **Ctrl + V** key combination.

E.3.9.3 Deep Copy

Deep Copy is the operation of copying all the data in a node.

The user selects the node to be copied and Right-Click to select **Deep Copy** or press **Ctrl + Shift + C**.

The user selects the location to save and Right-Click selects **Paste** or presses the **Ctrl + V** key combination.

E.3.9.4 Delete

Delete is the operation of deleting a node.

The user selects the node to be deleted and Right-Click to select **Delete** or press the **Del** key.

E.4. Configure runtime deployments

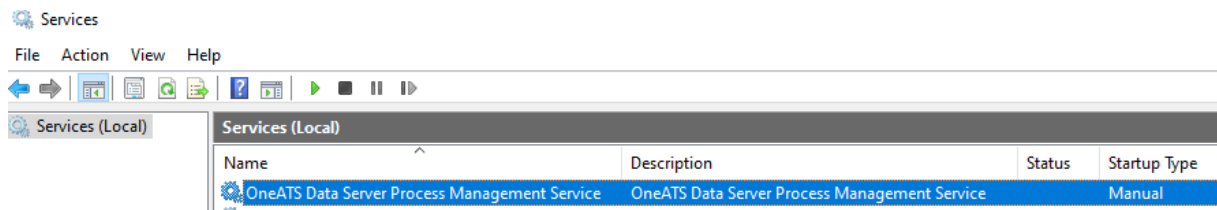
E.4.1. Configure DataServer services

Reference [E.2.3 Application Configuration](#)

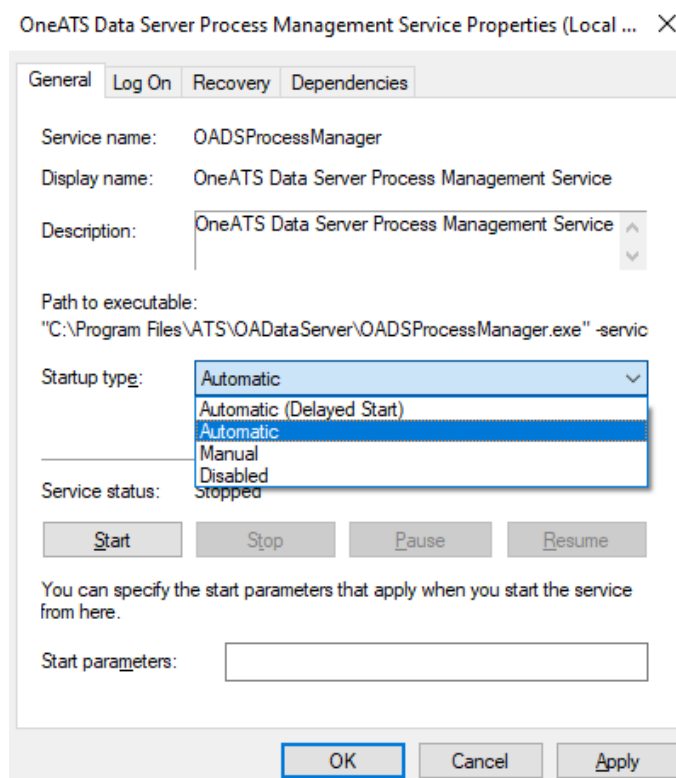
E.4.2. Boot OneATS DataServer service with Windows

To start OneATS DataServer service with Windows:

- Run the Window Services app

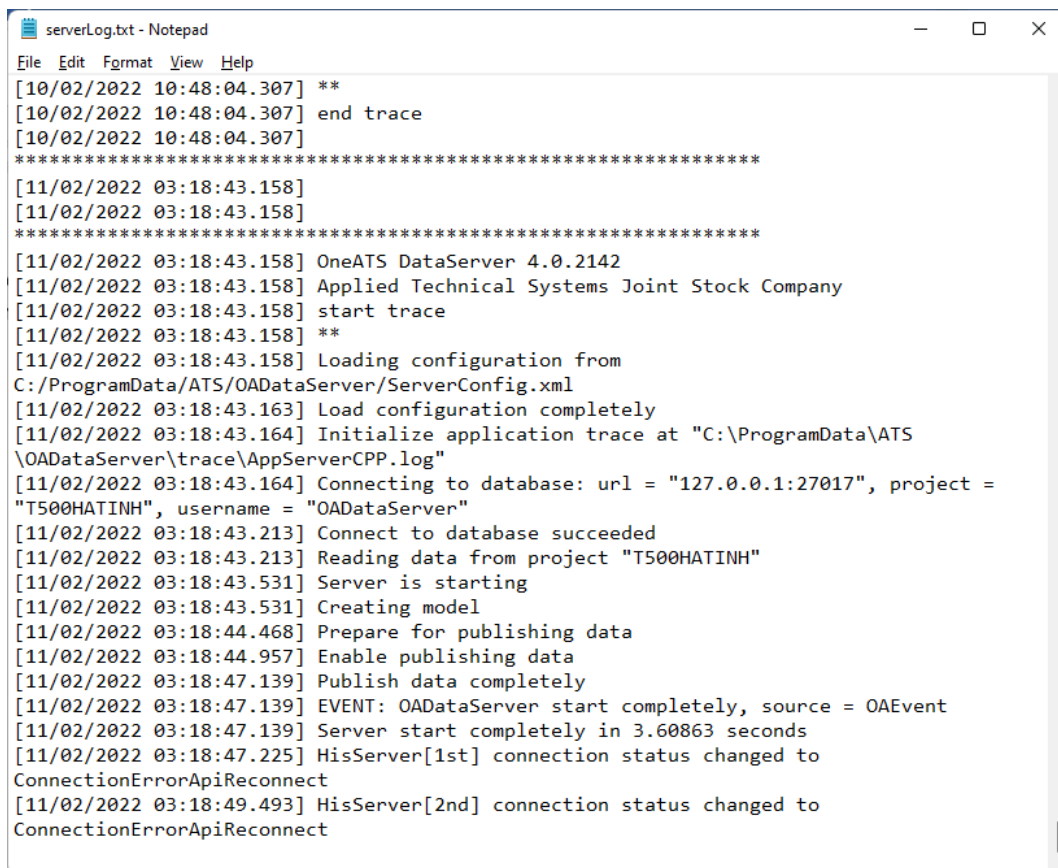


- Setting up OADDataServer service : Start-up Type = Automatic



E.5. Monitoring Log - runtime

The logs of the OneATS DataServer service are stored in the **serverLog.txt** file in the log configuration location.



```
serverLog.txt - Notepad
File Edit Format View Help
[10/02/2022 10:48:04.307] **
[10/02/2022 10:48:04.307] end trace
[10/02/2022 10:48:04.307]
*****
[11/02/2022 03:18:43.158]
[11/02/2022 03:18:43.158]
*****
[11/02/2022 03:18:43.158] OneATS DataServer 4.0.2142
[11/02/2022 03:18:43.158] Applied Technical Systems Joint Stock Company
[11/02/2022 03:18:43.158] start trace
[11/02/2022 03:18:43.158] **
[11/02/2022 03:18:43.158] Loading configuration from
C:/ProgramData/ATS/OADDataServer/ServerConfig.xml
[11/02/2022 03:18:43.163] Load configuration completely
[11/02/2022 03:18:43.164] Initialize application trace at "C:\ProgramData\ATS
\OADDataServer\trace\AppServerCPP.log"
[11/02/2022 03:18:43.164] Connecting to database: url = "127.0.0.1:27017", project =
"T500HATINH", username = "OADDataServer"
[11/02/2022 03:18:43.213] Connect to database succeeded
[11/02/2022 03:18:43.213] Reading data from project "T500HATINH"
[11/02/2022 03:18:43.531] Server is starting
[11/02/2022 03:18:43.531] Creating model
[11/02/2022 03:18:44.468] Prepare for publishing data
[11/02/2022 03:18:44.957] Enable publishing data
[11/02/2022 03:18:47.139] Publish data completely
[11/02/2022 03:18:47.139] EVENT: OADDataServer start completely, source = OAEvent
[11/02/2022 03:18:47.139] Server start completely in 3.60863 seconds
[11/02/2022 03:18:47.225] HisServer[1st] connection status changed to
ConnectionErrorApiReconnect
[11/02/2022 03:18:49.493] HisServer[2nd] connection status changed to
ConnectionErrorApiReconnect
```

Figure E-57. OneATS DataServer application log file

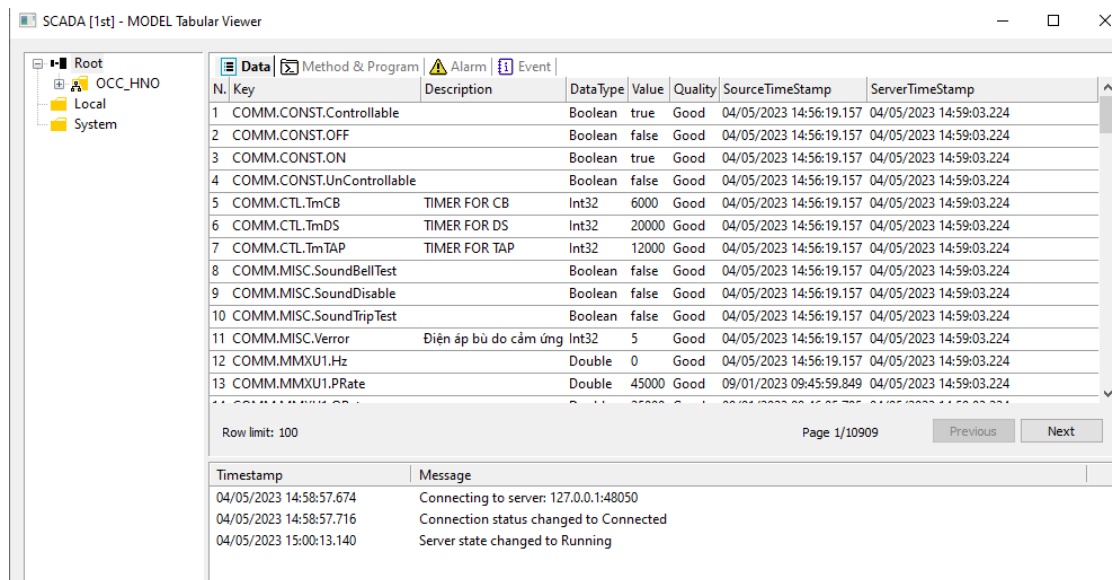
Basic Information:

- Load configuration file serverConfig.xml complete & configuration file error if any
- License error (if any)
- Connecting to Database / project name
- Redundancy status
- HIS Server Connection
- ...

E.6. Monitor runtime data

The "Tabular Viewer" tool allows users to monitor the real-time values of all data points in the system, perform value simulations and return real-time values for data points when the system is in operation mode.

On the menu bar, select **Model >> Tabular Viewer >> Select SCADA Server** to open **MODEL Tabular Viewer**.

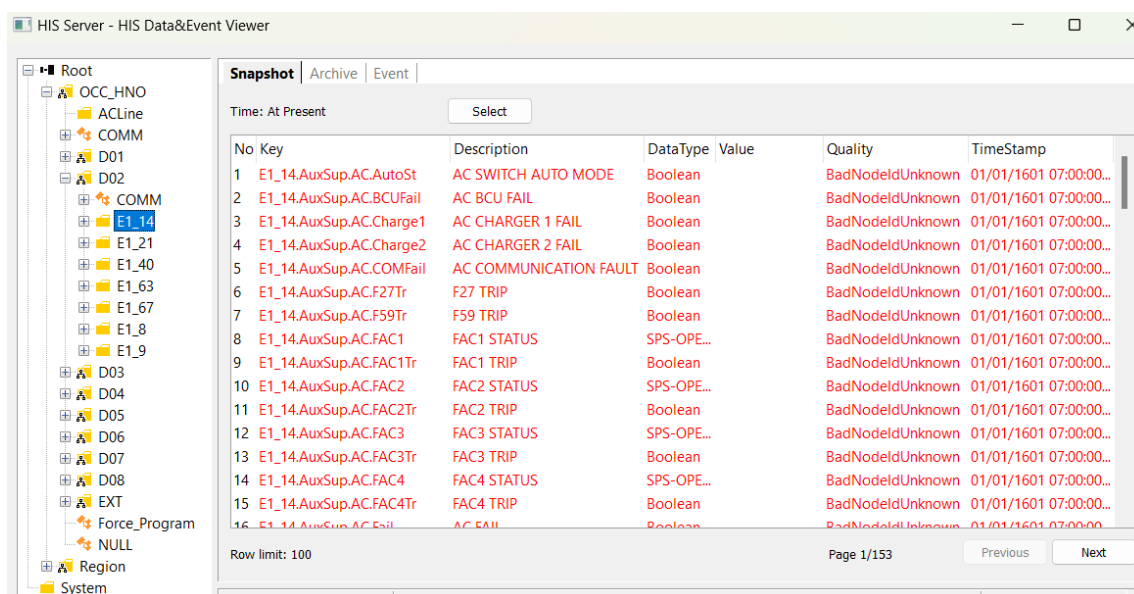


N	Key	Description	DataType	Value	Quality	SourceTimeStamp	ServerTimeStamp
1	COMM.CONST.Controllable		Boolean	true	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
2	COMM.CONST.OFF		Boolean	false	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
3	COMM.CONST.ON		Boolean	true	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
4	COMM.CONST.UnControllable		Boolean	false	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
5	COMM.CTL.TmCB	TIMER FOR CB	Int32	6000	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
6	COMM.CTL.TmDS	TIMER FOR DS	Int32	20000	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
7	COMM.CTL.TmTAP	TIMER FOR TAP	Int32	12000	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
8	COMM.MISC.SoundBellTest		Boolean	false	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
9	COMM.MISC.SoundDisable		Boolean	false	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
10	COMM.MISC.SoundTripTest		Boolean	false	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
11	COMM.MISC.Verror	Điện áp bù do cảm ứng	Int32	5	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
12	COMM.MMXU1.Hz		Double	0	Good	04/05/2023 14:56:19.157	04/05/2023 14:59:03.224
13	COMM.MMXU1.PRate		Double	45000	Good	09/01/2023 09:45:59.849	04/05/2023 14:59:03.224

Figure E-58. DataServer runtime data monitoring

In addition, the "Data & Event Viewer" tool allows users to monitor the value of data points in the HIS Server.

On the menu bar, select **HIS >> Data & Event Viewer**



No	Key	Description	DataType	Value	Quality	TimeStamp
1	E1_14.AuxSup.AC.AutoSt	AC SWITCH AUTO MODE	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
2	E1_14.AuxSup.AC.BCUFail	AC BCU FAIL	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
3	E1_14.AuxSup.AC.Charge1	AC CHARGER 1 FAIL	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
4	E1_14.AuxSup.AC.Charge2	AC CHARGER 2 FAIL	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
5	E1_14.AuxSup.AC.COMFail	AC COMMUNICATION FAULT	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
6	E1_14.AuxSup.AC.F27Tr	F27 TRIP	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
7	E1_14.AuxSup.AC.F59Tr	F59 TRIP	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
8	E1_14.AuxSup.AC.FAC1	FAC1 STATUS	SPS-OPE...		BadNodeIdUnknown	01/01/1601 07:00:00...
9	E1_14.AuxSup.AC.FAC1Tr	FAC1 TRIP	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
10	E1_14.AuxSup.AC.FAC2	FAC2 STATUS	SPS-OPE...		BadNodeIdUnknown	01/01/1601 07:00:00...
11	E1_14.AuxSup.AC.FAC2Tr	FAC2 TRIP	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
12	E1_14.AuxSup.AC.FAC3	FAC3 STATUS	SPS-OPE...		BadNodeIdUnknown	01/01/1601 07:00:00...
13	E1_14.AuxSup.AC.FAC3Tr	FAC3 TRIP	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
14	E1_14.AuxSup.AC.FAC4	FAC4 STATUS	SPS-OPE...		BadNodeIdUnknown	01/01/1601 07:00:00...
15	E1_14.AuxSup.AC.FAC4Tr	FAC4 TRIP	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...
16	E1_14.AuxSup.AC.Fail	AC FAIL	Boolean		BadNodeIdUnknown	01/01/1601 07:00:00...

Figure E-59. Data Monitoring on HISServer

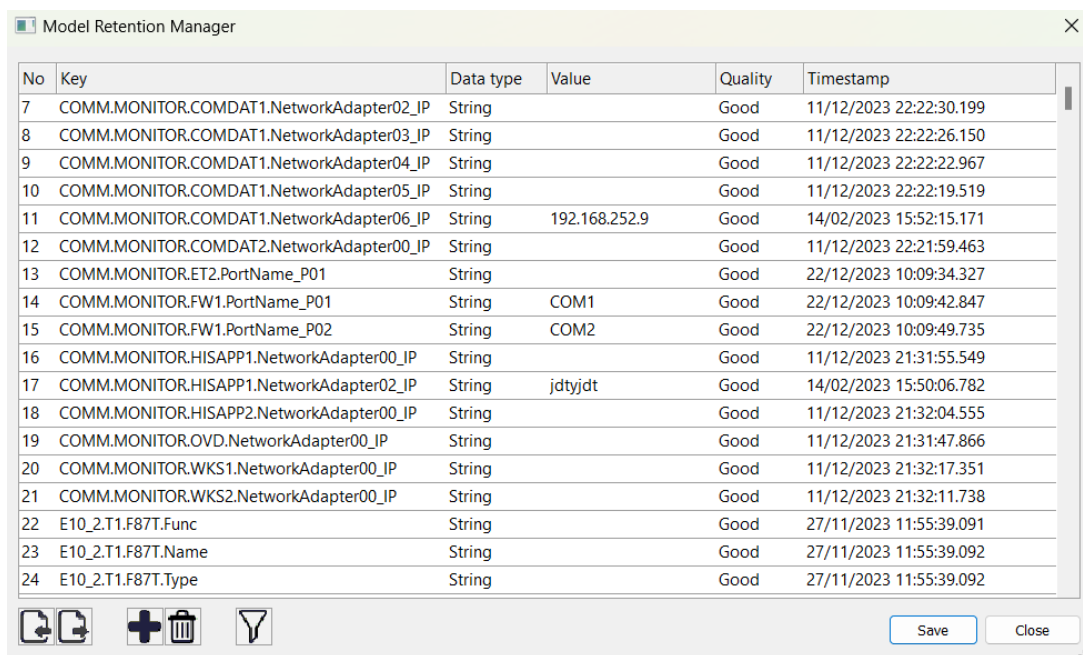
E.7. Switching Order Class

On the menu bar, select **Model >> Switching Order Class**

E.8. Data Retention Management

With the data management tool, Retention allows users to monitor the real-time value of data points configured as Retentive in the system.

On the menu bar, select **Model >> Retention Manager**.



No	Key	Data type	Value	Quality	Timestamp
7	COMM.MONITOR.COMDAT1.NetworkAdapter02_IP	String		Good	11/12/2023 22:22:30.199
8	COMM.MONITOR.COMDAT1.NetworkAdapter03_IP	String		Good	11/12/2023 22:22:26.150
9	COMM.MONITOR.COMDAT1.NetworkAdapter04_IP	String		Good	11/12/2023 22:22:22.967
10	COMM.MONITOR.COMDAT1.NetworkAdapter05_IP	String		Good	11/12/2023 22:22:19.519
11	COMM.MONITOR.COMDAT1.NetworkAdapter06_IP	String	192.168.252.9	Good	14/02/2023 15:52:15.171
12	COMM.MONITOR.COMDAT2.NetworkAdapter00_IP	String		Good	11/12/2023 22:21:59.463
13	COMM.MONITOR.ET2.PortName_P01	String		Good	22/12/2023 10:09:34.327
14	COMM.MONITOR.FW1.PortName_P01	String	COM1	Good	22/12/2023 10:09:42.847
15	COMM.MONITOR.FW1.PortName_P02	String	COM2	Good	22/12/2023 10:09:49.735
16	COMM.MONITOR.HISAPP1.NetworkAdapter00_IP	String		Good	11/12/2023 21:31:55.549
17	COMM.MONITOR.HISAPP1.NetworkAdapter02_IP	String	jdtyjdt	Good	14/02/2023 15:50:06.782
18	COMM.MONITOR.HISAPP2.NetworkAdapter00_IP	String		Good	11/12/2023 21:32:04.555
19	COMM.MONITOR.OVD.NetworkAdapter00_IP	String		Good	11/12/2023 21:31:47.866
20	COMM.MONITOR.WKS1.NetworkAdapter00_IP	String		Good	11/12/2023 21:32:17.351
21	COMM.MONITOR.WKS2.NetworkAdapter00_IP	String		Good	11/12/2023 21:32:11.738
22	E10_2.T1.F87T.Func	String		Good	27/11/2023 11:55:39.091
23	E10_2.T1.F87T.Name	String		Good	27/11/2023 11:55:39.092
24	E10_2.T1.F87T.Type	String		Good	27/11/2023 11:55:39.092

Figure E-60. Data Retention Management

Retention data points can be imported, exported from/out of excel files in .xlsx format or inserted, deleted, filtered.

E.9. Programming in OneATS DataServer

OneATS DataServer support scripts (code snippets) to meet flexible needs in data processing.

E.9.1. Types of Scripts

Type	Execution conditions
Mapping – script type	Auto-execute, when the monitored variables – Monitored Item – change
Method	Called by Client
Programs	Automatic execution depending on the configuration, executed under the condition that the monitored variables – Monitored Item – change or periodically over time
Function script	Called by Method/Program/Mapping – other scripts/Functions

E.9.2. Scripting Languages in OneATS DataServer

OneATS DataServer using the Lua programming language, in combination with **Object**, **Variable**, **Method**, **Program** defined from the Data Model and built-in functions to create a flexible and efficient programming environment.

E.9.3. Calling Method

Method is called by a script:

Syntax:

```
Method:Call(parameters)
```

Note: When typing script content, use CTRL + Space to see the suggested functions

Example: MODEL::D 01. XCBR1. PosCtl:Call(1)

E.9.4. Variables in the script

Variable Type	Describe	Operating Range
System variables	System Default Variables. Listed in Model/Configuration/Local Variables	In the whole application
Local application variables	Declared in the MODEL/Configuration/Local Variables menu	In the whole application
Local script variables	Declared in the script	Inside the script
DataModel/ Variable.dotfield	The variables are declared from the Data Model. The DataModel/Variable is defined by the key of the variable.	In the whole application

Objects can be declared and used as array objects/lists.

E.9.5. Data Model/Variable dotfield

Dotfield	Data Type	Describe	Syntax / Example
.Value	Variable type (bool, int, ...)	DataModel/Variable Values	[Key].value Example: E10::IntVar.value
.Quality	OAQuality (uint32)	Quality of DataModel/Variable The base data type is uint32, with a set of values that follow the OPC UA standard	[Key].quality Example: E10::IntVar.quality
.Timestamp	OADatetime (uint64)	DataModel/Variable Timestamp The base data type is uint64. Reference landmarks: 0h 0p 0.000s 1/1/1601 One unit corresponds to 100ns	[Key].timestamp Example: E10::IntVar.timestamp

E.9.6. Built-in function

E.9.6.1.1 Min()

Definition: the calculation returns the smallest value in the value set.

Syntax:

```
Min(number1,number2, ...)
```

Example: MODEL::IntVar:Update(Min(2,3,4))

The result of the variable MODEL::IntVar has a value = 2.

E.9.6.1.2 Max()

Definition: the calculation returns the largest value in the value set.

Syntax:

```
Max(number1,number2, ...)
```

Example: MODEL::IntVar:Update(Max(2,3,4))

The result of the variable MODEL::IntVar has a value = 4.

E.9.6.1.3 IsGood()

Definition: returns true value if quality good

Syntax:

```
IsGood(quality)
```

Example: MODEL::BoolVar:Update(IsGood(Var1.quality))

If Var1.quality is good, the MODEL::BoolVar variable has a value of true.

E.9.6.1.4 IsNotGood()

Definition: return a true value if the quality is not good.

Syntax:

```
IsNotGood(quality)
```

E.9.6.1.5 IsBad()

Definition: returns a true value if the quality is bad.

Syntax:

```
IsBad(quality)
```

E.9.6.1.6 IsNotBad()

Definition: returns a true value if the quality is not bad.

Syntax:

```
IsNotBad(quality)
```

E.9.6.1.7 GetQualityString()

Definition: return the chain represents quality.

Syntax:

```
GetQualityString(quality)
```

For example, MODEL::Str:Update(GetQualityString(MODEL::Var.quality)).

If Var.quality = 0, Str.value = "Good"; Var.quality = 80320000 (hexa),
Str.value = "BadWaitingForInitialData".

E.9.6.1.8 ReferTo()

Definition: Reference to an object or variable

Syntax:

```
ReferTo(Key)
```

Example

```
local p=0
```

```
for i=1,10,1 do
```

```
    local pRef= ReferTo("RTS. RT"... i..". totW")
```

```
    if pRef~=nil then p = p + pRef.value end
```

```
End
```

```
MODEL::RTS_P:Update(p)
```

Result: RTS_P is the sum of the RTS variables. RT1.totW, RTS. RT2.totW,..
RTS. RT10.totW

E.9.6.1.9 SetBit()

Definition: set the bit in the given position to 1.

Syntax:

```
SetBit(number,n)
```

For example, MODEL::Var1:Update(SetBit(MODEL::IntVar.value,1)). Suppose the
variable IntVar.value = 4, after the command on the variable Var1.value = 6.

E.9.6.1.10 ClearBit()

Definition: set the bit to a certain position with a value of 0.

Syntax:

```
ClearBit(number,n)
```

For example, MODEL::Var1:Update(SetBit(MODEL::IntVar.value,1)). Suppose the variable IntVar.value = 6, after the command on the variable Var1.value = 4.

E.9.6.1.11 ToggleBit()

Definition: Reverse the bit value at a given location.

Syntax:

```
ToggleBit(number,n)
```

For example, MODEL::Var1:Update(ToggleBit(MODEL::IntVar.value,1)). Suppose the variable IntVar.value = 6, after the command on the variable Var1.value = 4.

E.9.6.1.12 CheckBit()

Definition: Returns a bit value at a given location.

Syntax:

```
CheckBit(number,n)
```

For example, MODEL::Var1:Update(CheckBit(6,1)). Var1.value variable = 1.

E.9.6.1.13 Force()

Definition: Write a manual value for a point.

Syntax:

```
Force(Key,value)
```

For example: Force(MODEL::D 01. XCBR1. Str.value,1)

E.9.6.1.14 ForceGood()

Definition: Write a manual value for a point with good quality.

Syntax:

```
ForceGood(Key,value)
```

For example: Force(MODEL::D 01. XCBR1. Str.value,1), variable MODEL::D 01. XCBR1. Str.value=1 and MODEL::D 01. XCBR1. Str.quality is Good

E.9.6.1.15 Unforce()

Definition: Update the actual value for a variable that has been manually valued.

Syntax:

```
Unforce(Key)
```

Example: `Unforce("MODEL::D 01. XCBR1. Str.value")`.

E.9.6.1.16 UnforceAll()

Definition: Switch back to real value update mode with all variables that have been switched to force mode.

Syntax:

```
UnforceAll()
```

E.9.6.1.17 EnableAlarm()

Definition: Allows the alarm of a point.

Syntax:

```
EnableAlarm(Key)
```

For example, `EnableAlarm("MODEL::D 01. XCBR1. PosSt")`.

E.9.6.1.18 DisableAlarm()

Definition: Prohibiting the occurrence of an alarm of a point.

Syntax:

```
DisableAlarm(Key)
```

E.9.6.1.19 GetAlarmLimit()

Definition: returns the status of the limit alarm corresponding to the point (LoRsn: -3, LoLo: -2, Lo: -1, None: 0, Hi: 1, HiHi: 2, HiRsn: 3).

Syntax:

```
GetAlarmLimit(Key)
```

E.9.6.1.20 ChangeTemplateInputText()

Definition: Update Input Text for an instance template object

Syntax:

ChangeTemplateInputText(Key,inputId,inputText)

For example, `ChangeTemplateInputText("D01","BayName","171")`: **BayName** in Instance D01 is replaced with 171.

E.9.6.1.21 ChangeTemplateInputConstant()

Definition: Update Input Constant for an instance template object

Syntax:

ChangeTemplateInputConstant(Key,inputId,inputConstant)

Example: `ChangeTemplateInputConstant("BB51","PPVmax_Lo",Var2.value)`

E.9.6.1.22 GetControllingUser()

Definition: Returns a string that is the name of the user performing the control.

Syntax:

GetControllingUser()

E.9.6.1.23 GetUserAccessLevel()

Definition: Returns the access level value of the user performing the control.

Syntax:

GetUserAccessLevel()

E.9.6.1.24 GetSystemTime()

Definition: Returns a number that represents the date of the system.

Syntax:

GetSystemTime()

E.9.6.1.25 GetTimeFromString()

Definition: Convert a string to time.

Syntax:

GetTimeFromString(StrDateTime)

Example: `GetTimeFromString("05/05/05 11:15")` → 1.32*1017.

E.9.6.1.26 DateTimeToString()

Definition: Convert time to a string of format "%d/%m/%Y %H:%M:%d"

Syntax:

```
DateTimeToString(dt)
```

E.9.6.1.27 EnableTimer()

Definition: Start a timer.

Syntax:

```
Timer:Enable()
```

For example, LOCAL::CBTimer:Enable().

E.9.6.1.28 ResetTimer()

Definition: sets the value of the timer's Enabled variable to false and ends the timer's count.

Syntax:

```
Timer:Reset()
```

Example: LOCAL::CBTimer:Reset()

E.9.6.1.29 Update()

Definition: Update value, quality, timestamp for point.

Syntax:

```
OutVar:Update(InVar)
```

For example, after the Var1:Update(Var2) statement we have:

- Var1.value = Var2.value
- Var1.quality = Var2.quality
- Var1.timestamp = Var2.timestamp

E.9.6.1.30 ResetLatch()

Definition: Latch removal for a latched element. If the element is a variable, that variable is latch-dissolved; If the verb is an object, all variables in the object are latch-dissolved.

Syntax:

```
ElementLatched:ResetLatch()
```

Example: MODEL::Var1:ResetLatch()

E.9.6.1.31 ReferTo()

Definition: Refer to object or variable

Syntax:

```
ReferTo(keyname)
```

Example

```
local p=0
for i=1,10,1 do
    local pRef= ReferTo("RTS. RT"... i..". totW")
    if pRef~=nil then p = p + pRef.value end
end

MODEL::RTS_P:Update(p)
```

Result: RTS_P is the total value of variables. RTS. RT1.totW, RTS. RT2.totW, ... RTS. RT10.totW

E.9.7. Using arrays in programming

Objects can be declared and used as array/list objects.

For details of functions and uses, please refer to [Programming in Lua \(first edition\)](#)

E.9.7.1 Create a new array

- ➔ Create an object on the Data Model, for example MODEL::QNA_PC. RT
- ➔ Object MODEL::QNA_PC. RT is used as 1 array to save objects (for example, objects in the form of QNA_RT001, QNA_RT002,...)
- ➔ Empty array initialization syntax:

```
ArrayName= {}
```

Example : MODEL::QNA_PC. RT={}

- Result : set the array to an empty array

E.9.7.2 Functions used with arrays

E.9.7.2.1 Add elements to an array

Syntax

```
table.insert(ArrayName,ArrayElement)
```

Example 1 :

```
table.insert(MODEL::QNA_PC. RT,MODEL::QNA_RT001)
```

- Result: The MODEL::QNA_RT001 object is added to the array

Example 2 :

```
for i=1,9,1 do
```

```
local pRef = ReferTo("QNA_RT00"... s)
if pRef ~= nil then table.insert(MODEL::QNA_PC. RT,pRef) end
End
```

- Result: objects from MODEL::QNA_RT001 MODEL::QNA_RT009 are added to arrays→

E.9.7.2.2 Number of array elements

Syntax

#ArrayName

Example:

```
local n
n = # MODEL::QNA_PC. RT
```

- Result : n returns the number of elements of the array

E.9.7.2.3 Access array elements

Syntax

ArrayName[i]

- Example:

```
n = # MODEL::QNA_PC. RT
totW=0
for i=1,n,1 do
    totW = MODEL::QNA_PC. RT[i]. POI.totW.value + totW
End
```

Result : totW gets the sum value of POI.totW variables of elements in the array



Contact

ATS Building

74 Ba Trieu Str., Hanoi, Vietnam

T. +84-24-3825 1072

F. +84-24-3825 8037

W. www.ats.com.vn

E. contact@ats.com.vn

